TODAY'S AGENDA

B+Tree Overview

Design Choices

Optimizations

B-TREE FAMILY

There is a specific data structure called a **B-Tree**.

People also use the term to generally refer to a class of balanced tree data structures:

→ **B-Tree** (1970)
→ **B+Tree** (1973)
→ **B*Tree** (1977?)
→ **B^{link}-Tree** (1981)
→ **Bε-Tree** (2003)
→ **Bw-Tree** (2013)

B+TREE

A **B+Tree** is a self-balancing, ordered ***m***-way tree for searches, sequential access, insertions, and deletions in **$O(\log_m n)$** where ***m*** is the tree fanout.

→ It is perfectly balanced (i.e., every leaf node is at the same depth in the tree)

→ Every node other than the root is at least half-full

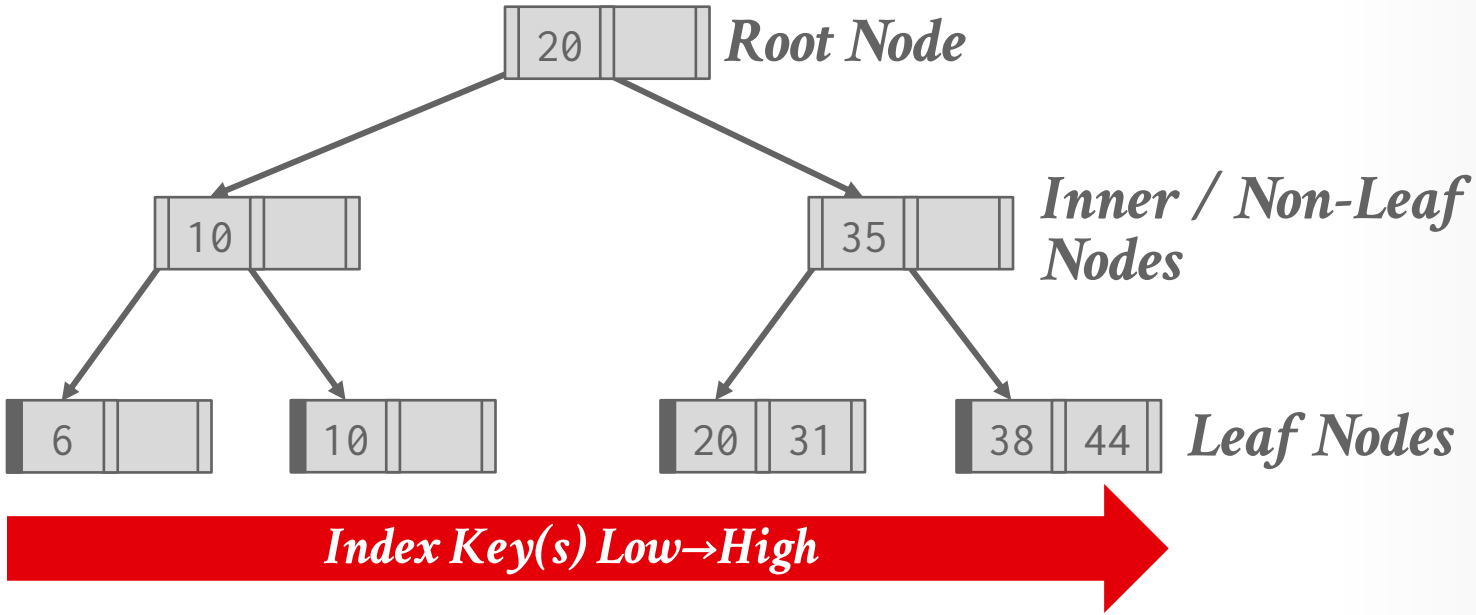
`m/2-1 ≤ #keys ≤ m-1`

→ Every inner node with **k** keys has **k+1** non-null children.

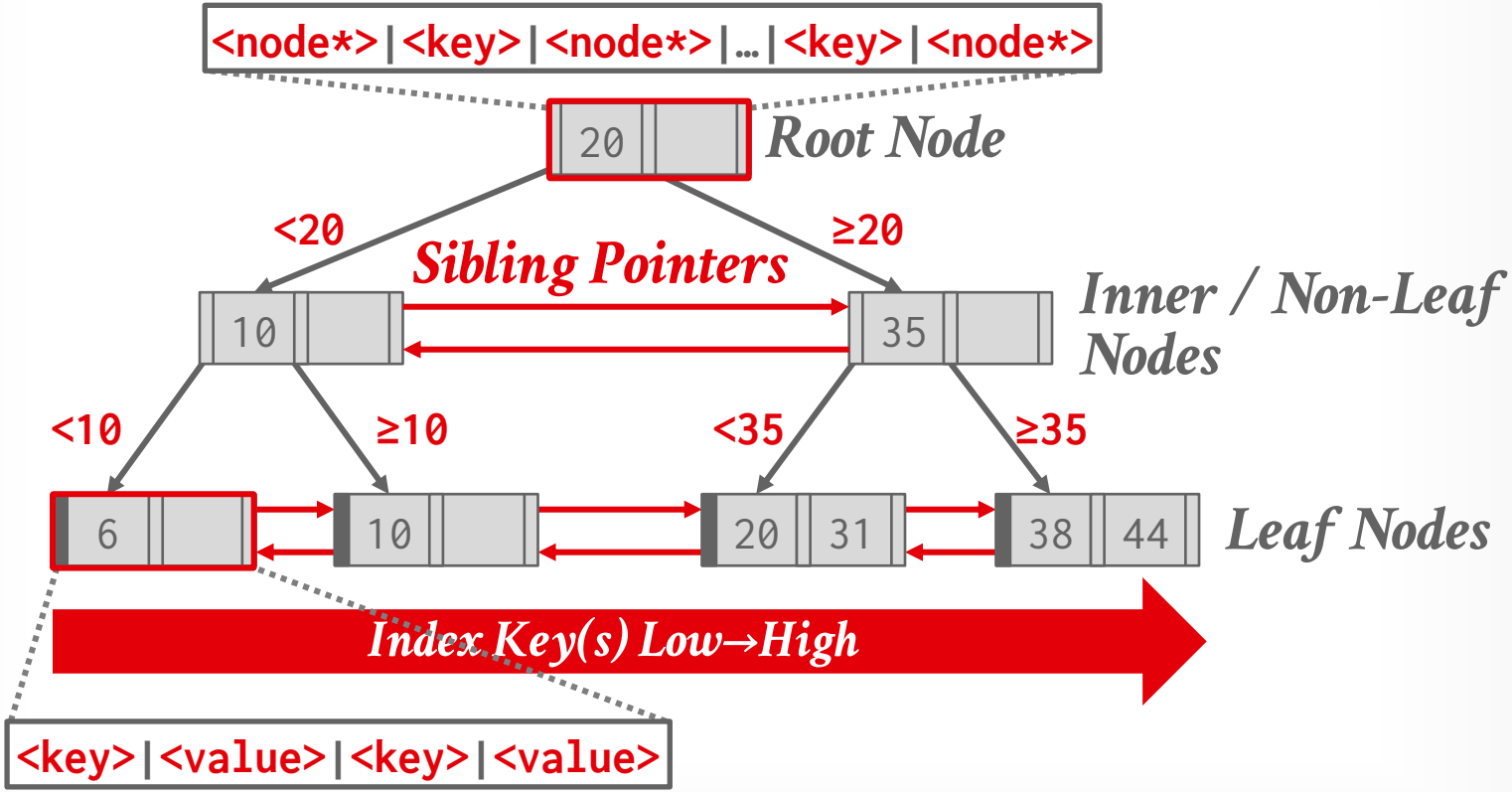
→ Optimized for reading/writing large data blocks.

Some real-world implementations relax these properties, but we will ignore that for now…

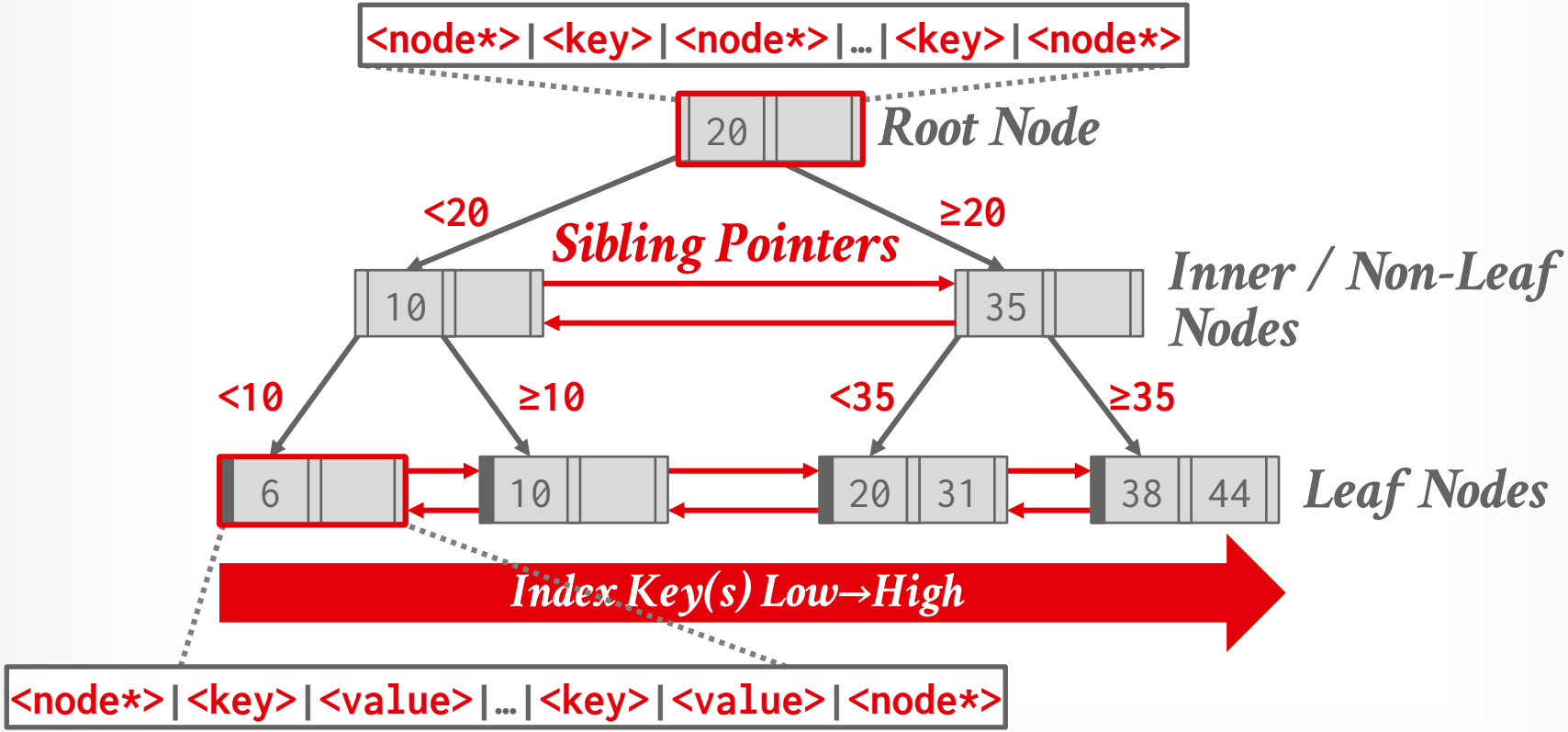
B+TREE EXAMPLE



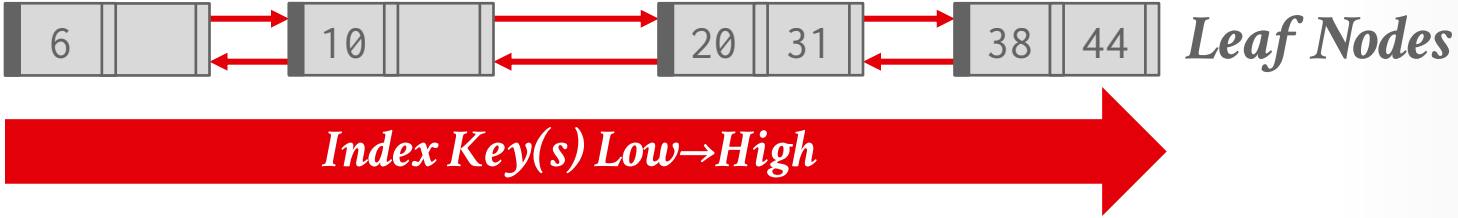
B+TREE EXAMPLE



B+TREE EXAMPLE



B+TREE EXAMPLE



NODES

Every B+Tree node is comprised of an array of key/value pairs.

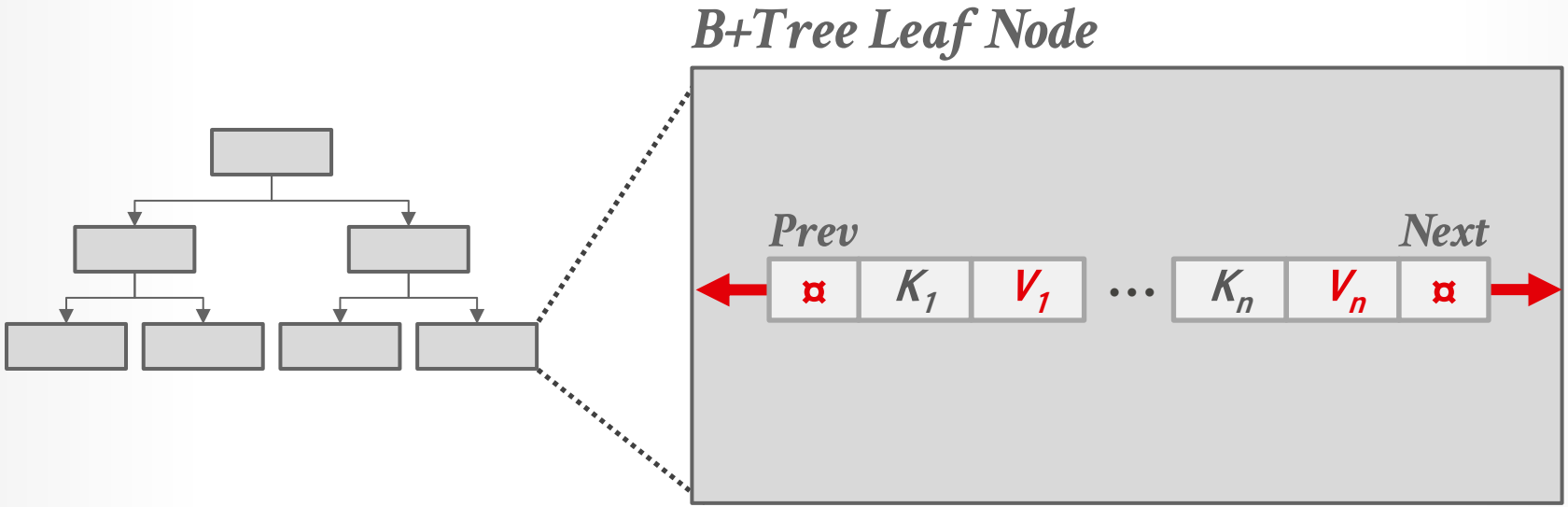
→ The keys are derived from the index's target attribute(s).

→ The values will differ based on whether the node is classified as an **inner node** or a **leaf node.**

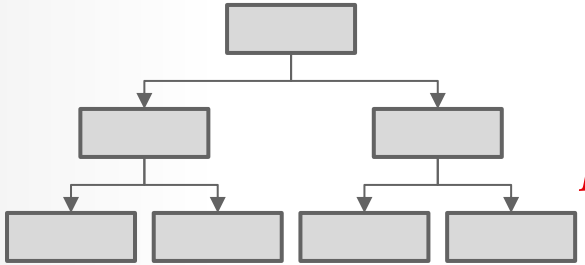
The arrays are (usually) kept in sorted key order.

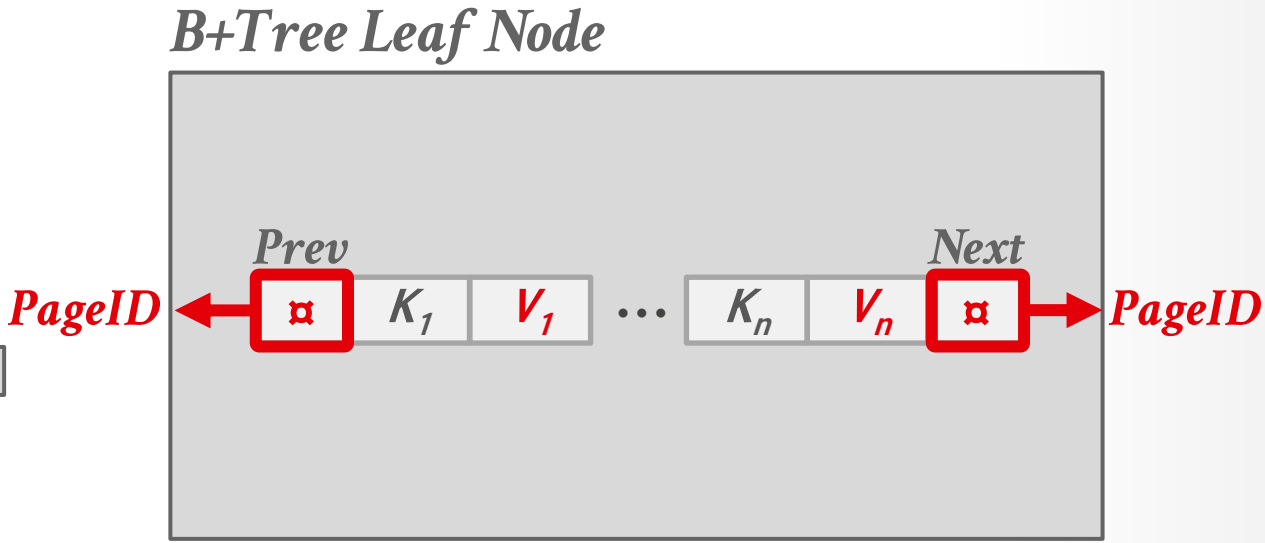
Store all **NULL** keys at either first or last leaf nodes.

B+TREE LEAF NODES

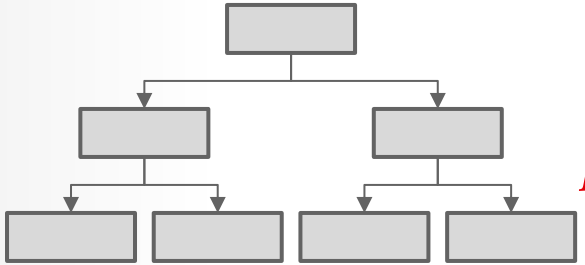


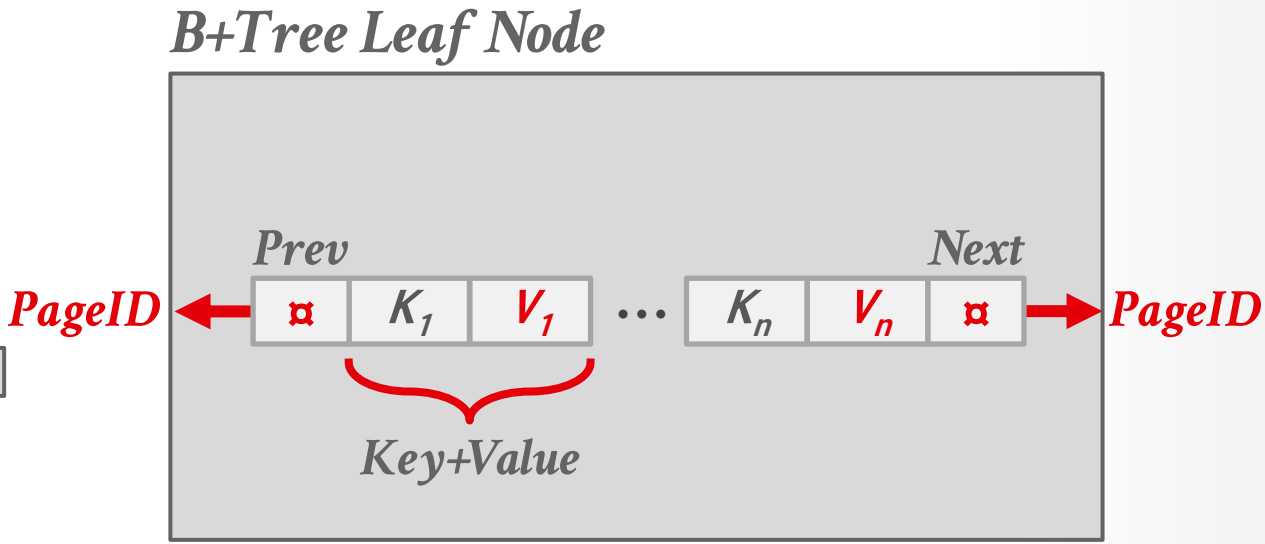
B+TREE LEAF NODES



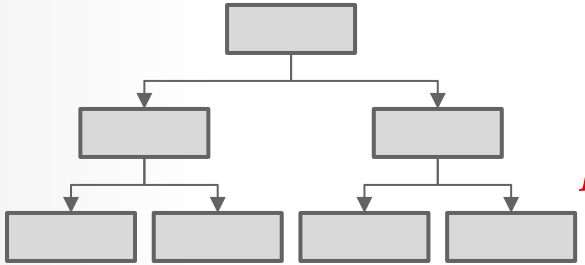


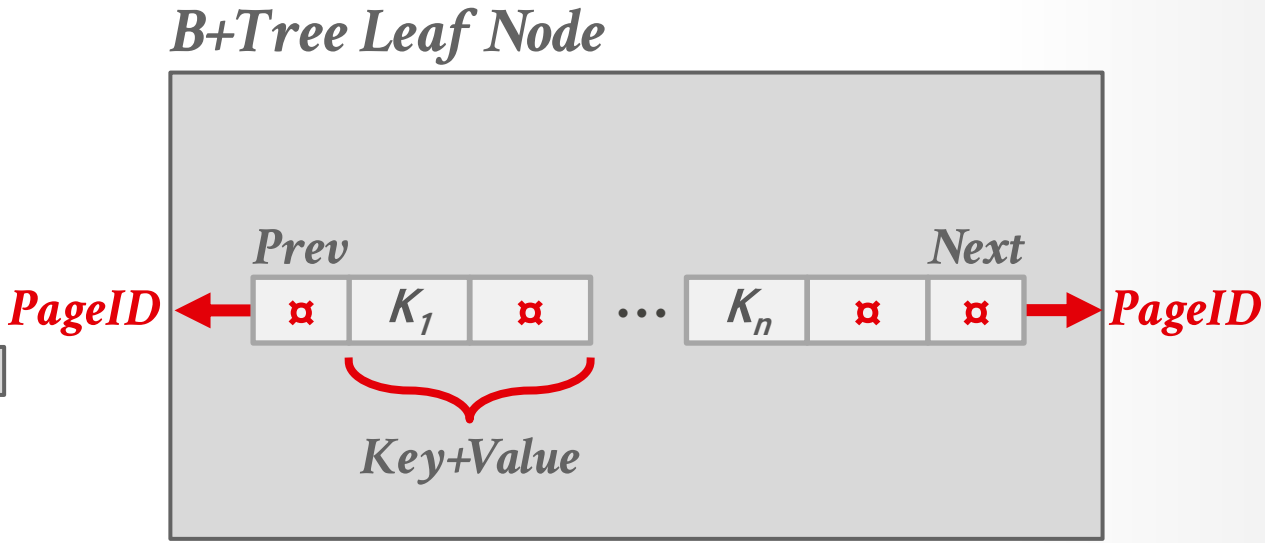
B+TREE LEAF NODES



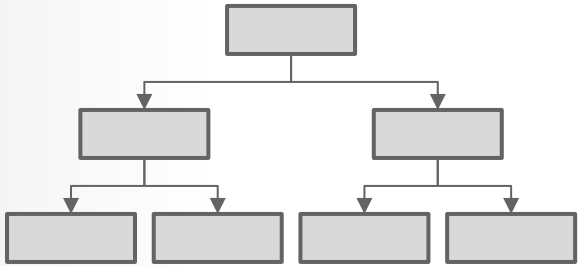


B+TREE LEAF NODES





B+TREE LEAF NODES



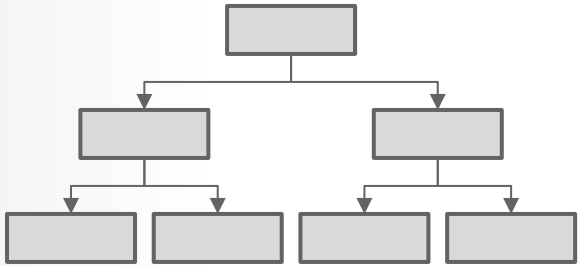
B+Tree Leaf Node

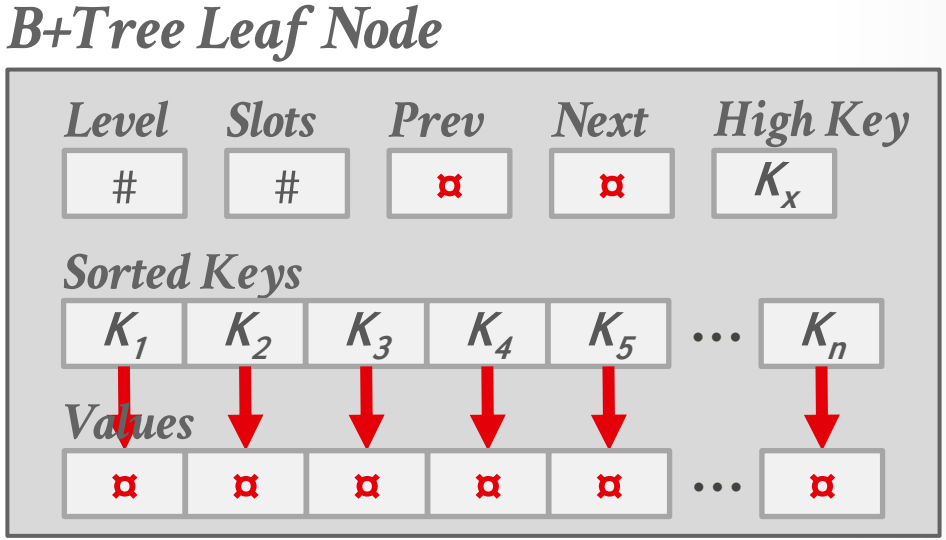
Level	Slots	Prev	Next	High Key
#	#	¤	¤	K_x

Sorted Key/Value Pairs

K_1	¤	K_2	¤	K_3	¤
K_4	¤	K_5	¤	…	

B+TREE LEAF NODES





LEAF NODE VALUES

Approach #1: Record IDs

→ A pointer to the location of the tuple to which the index entry corresponds.
→ Most common implementation.

PostgreSQL
Microsoft SQL Server
IBM DB2
ORACLE

Approach #2: Tuple Data

→ Index-Organized Storage (Lecture #05)
→ Primary Key Index: Leaf nodes store the contents of the tuple.
→ Secondary Indexes: Leaf nodes store tuples' primary key as their values.

SQLite
Microsoft SQL Server
MySQL
ORACLE

B-TREE VS. B+TREE

The original **B-Tree** from 1971 stored keys and values in all nodes in the tree.

→ More space-efficient, since each key only appears once in the tree.

A **B+Tree** only stores values in leaf nodes. Inner nodes only guide the search process.

B+TREE: INSERT

Find correct leaf node **L**.
Insert data entry into **L** in sorted order.

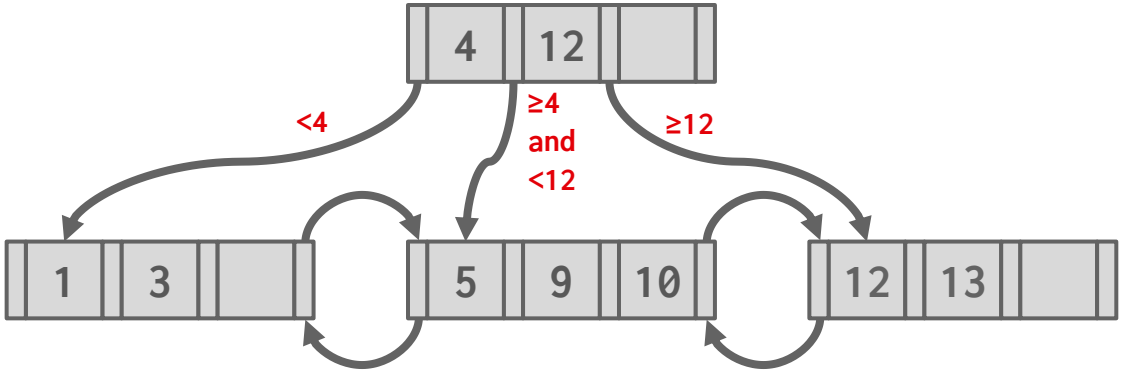
If **L** has enough space, done!

Otherwise, split **L** keys into **L** and a new node L_2
→ Redistribute entries evenly, copy up middle key.
→ Insert index entry pointing to L_2 into parent of **L**.

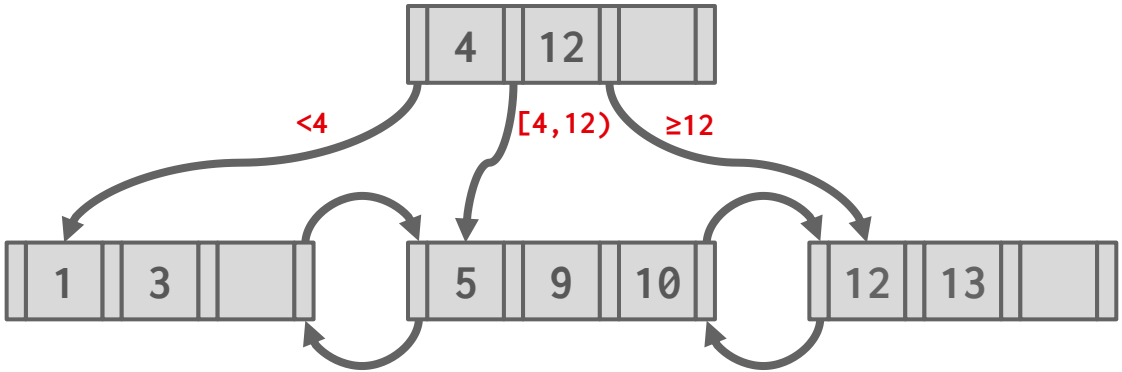
To split inner node, redistribute entries evenly, but push up middle key.

Source: Chris Re

B+TREE: INSERT EXAMPLE (1)

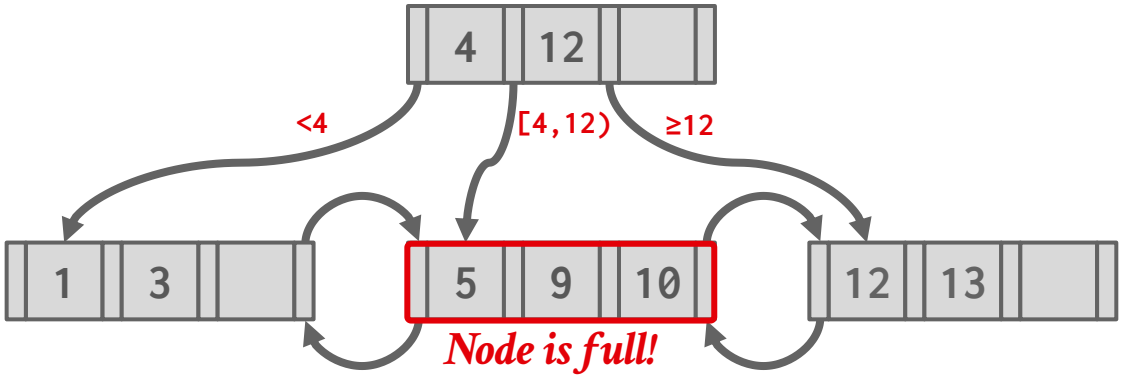


B+TREE: INSERT EXAMPLE (1)



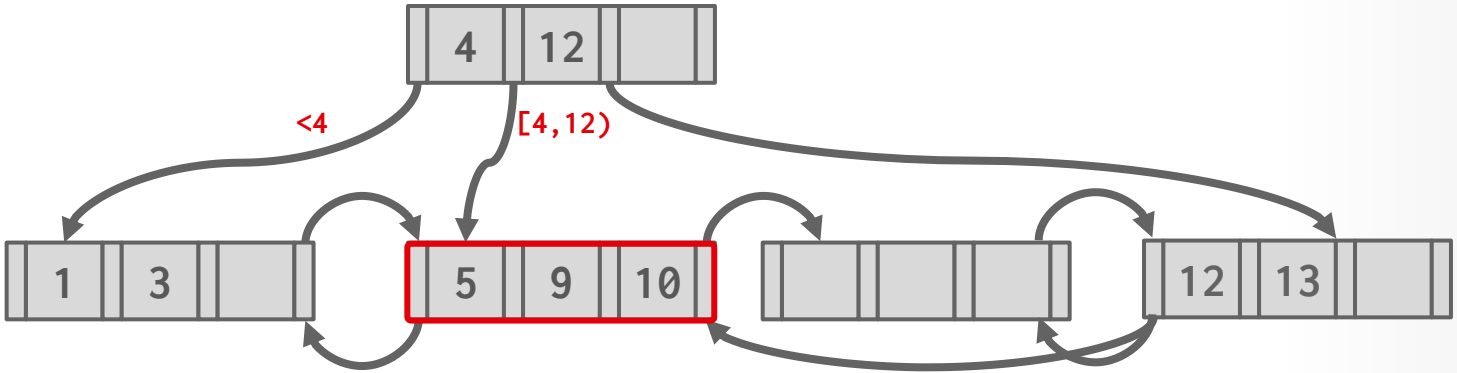
B+TREE: INSERT EXAMPLE (1)

Insert 6



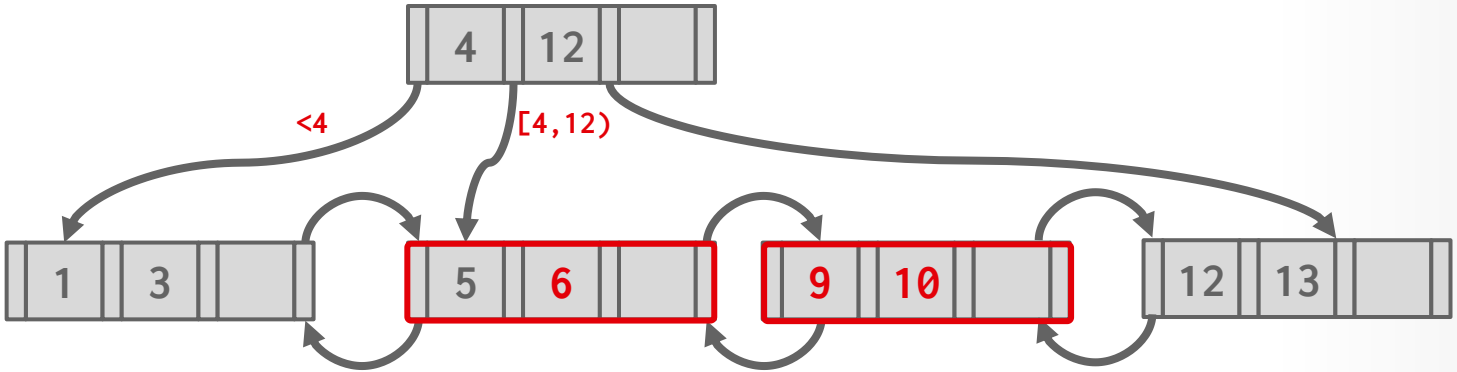
B+TREE: INSERT EXAMPLE (1)

Insert 6



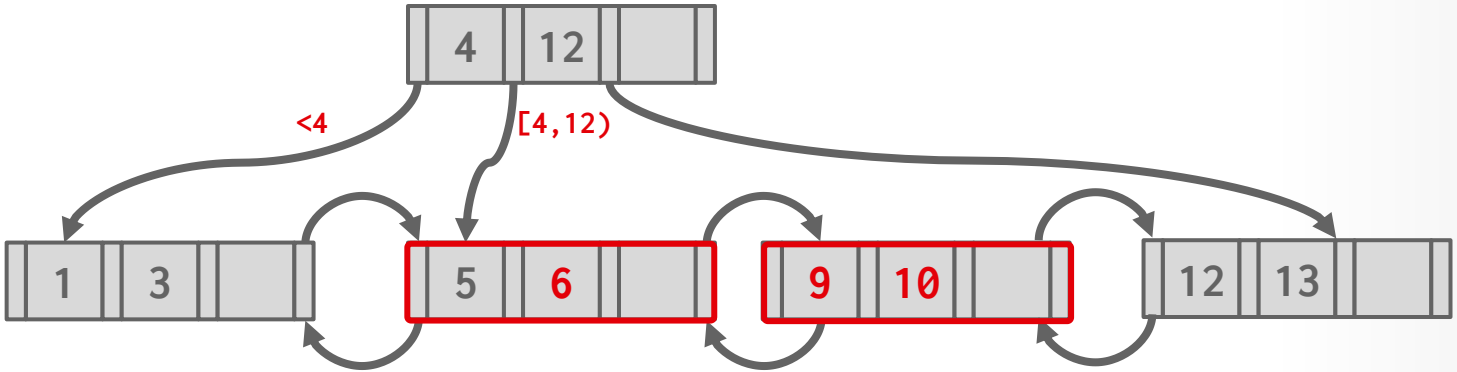
B+TREE: INSERT EXAMPLE (1)

Insert 6



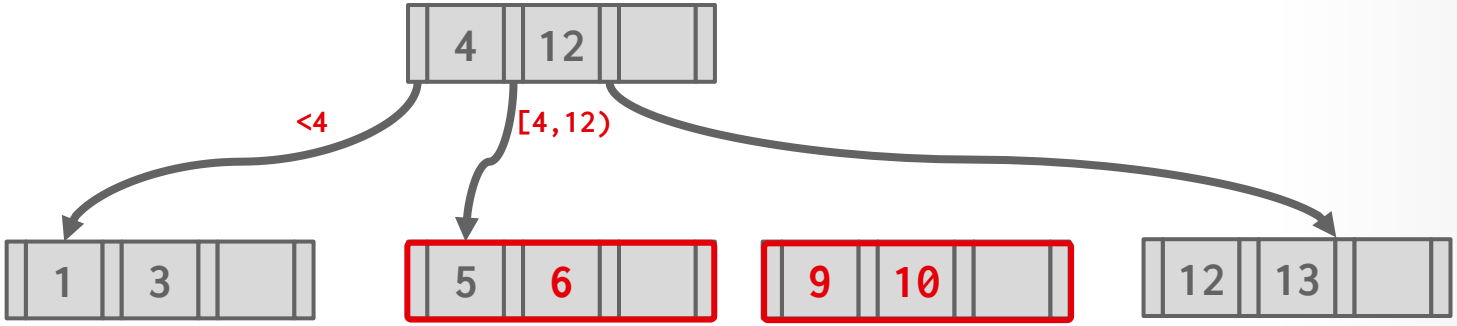
B+TREE: INSERT EXAMPLE (1)

Insert 6



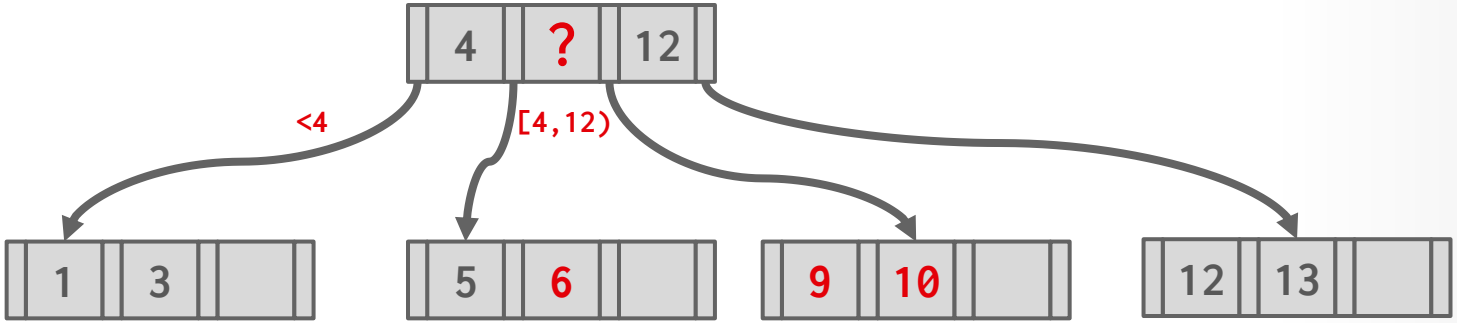
B+TREE: INSERT EXAMPLE (1)

Insert 6



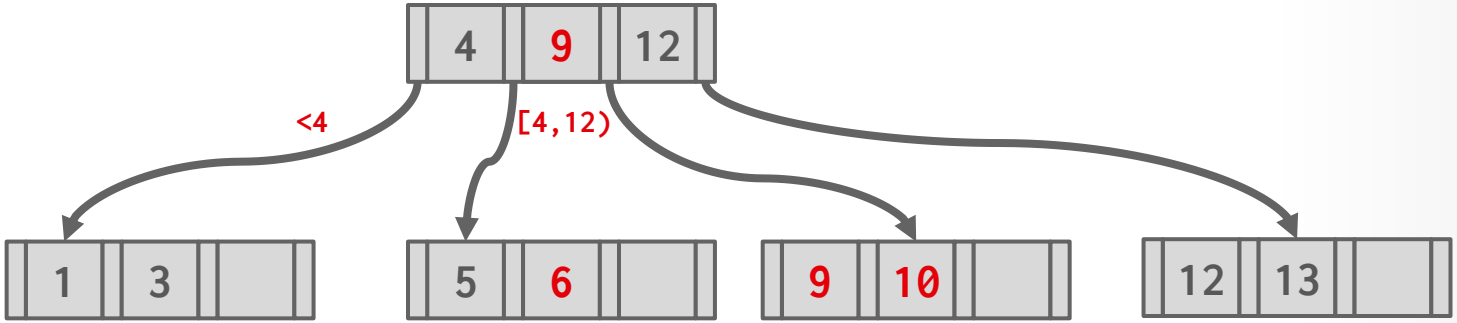
B+TREE: INSERT EXAMPLE (1)

Insert 6



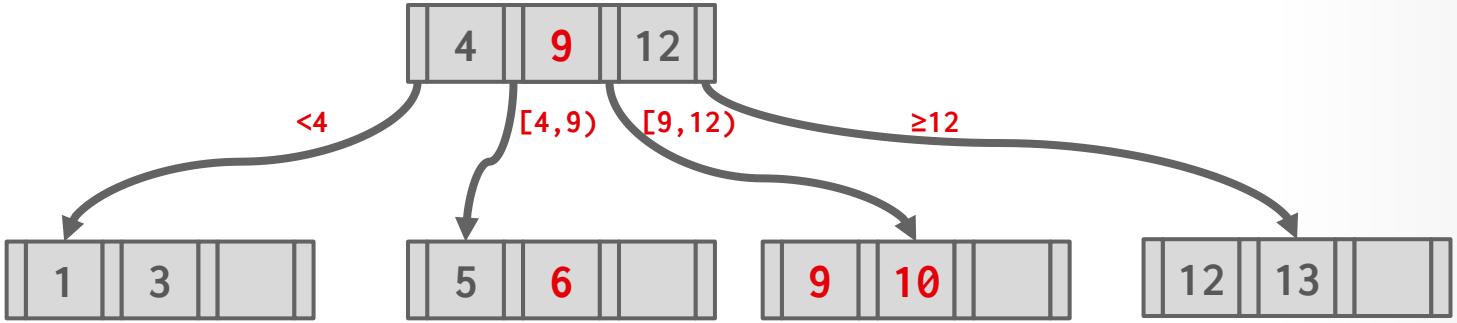
B+TREE: INSERT EXAMPLE (1)

Insert 6



B+TREE: INSERT EXAMPLE (1)

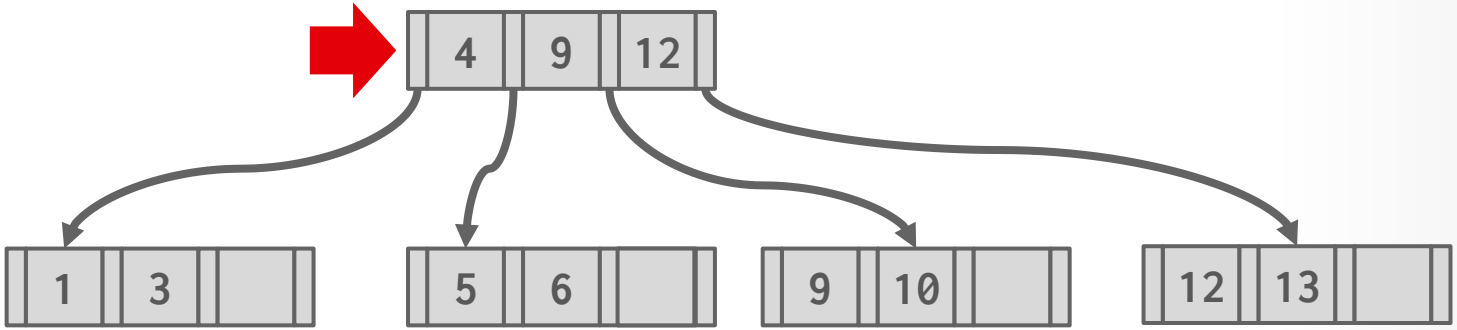
Insert 6



B+TREE: INSERT EXAMPLE (1)

Insert 6

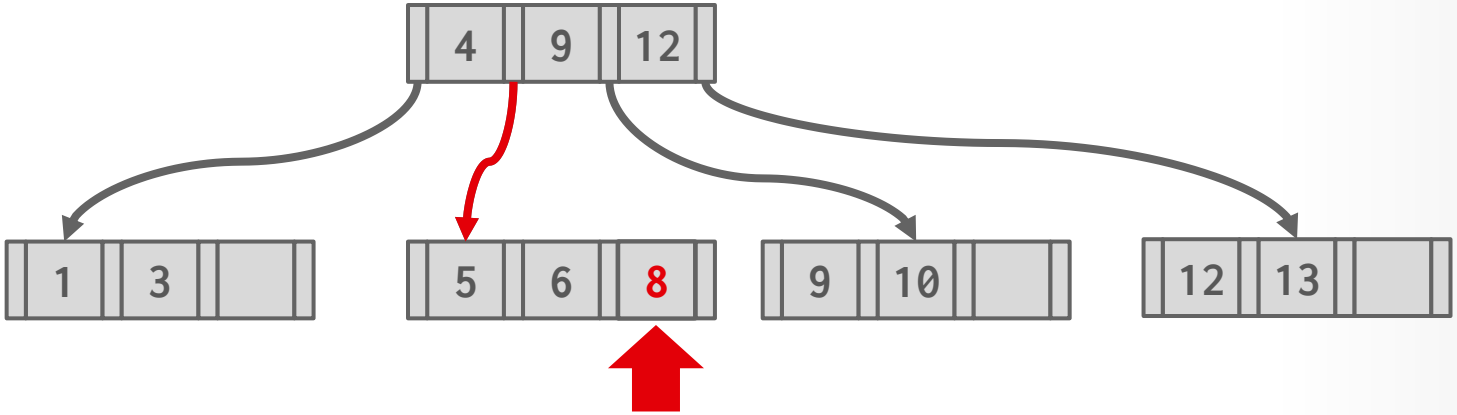
Insert 8



B+TREE: INSERT EXAMPLE (1)

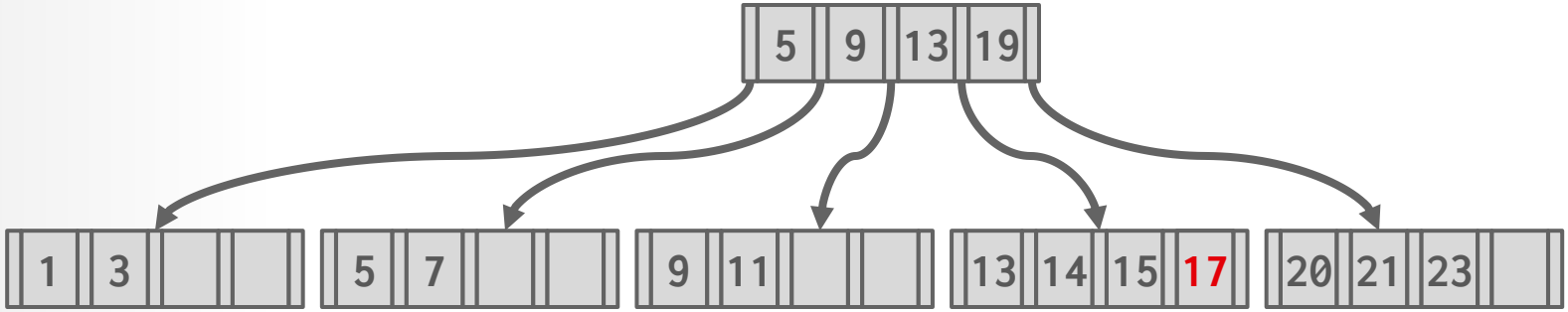
Insert 6

Insert 8



B+TREE: INSERT EXAMPLE (2)

Insert 17

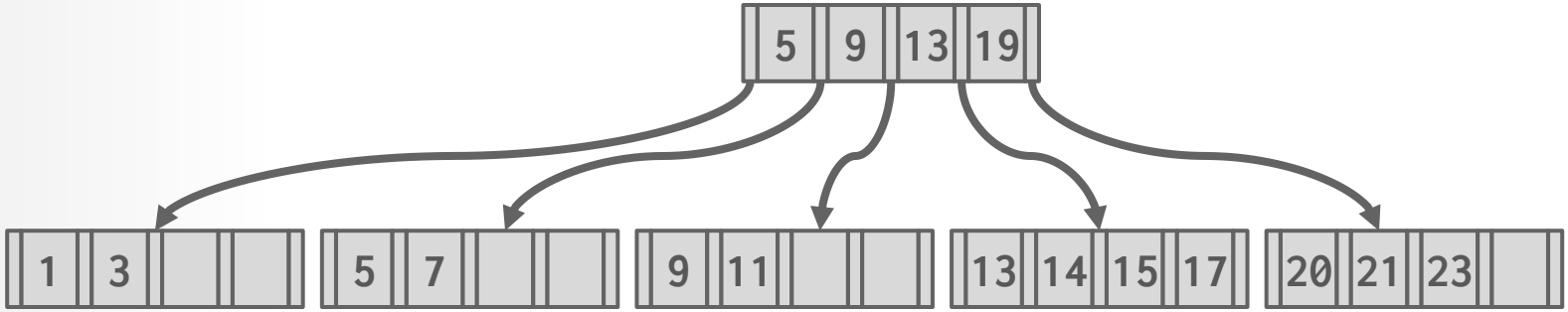


Note: New Example/Tree.

B+TREE: INSERT EXAMPLE (3)

Insert 17

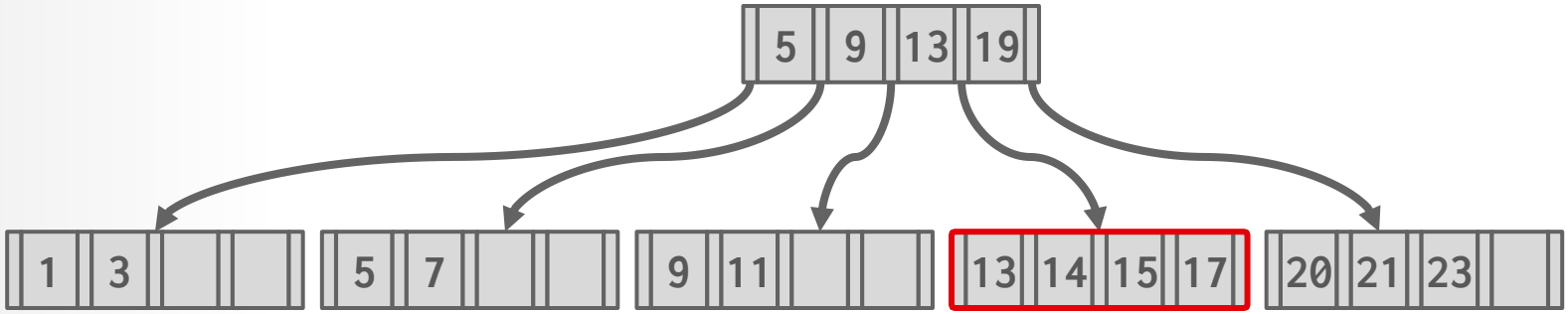
Insert 16



B+TREE: INSERT EXAMPLE (3)

Insert 17

Insert 16

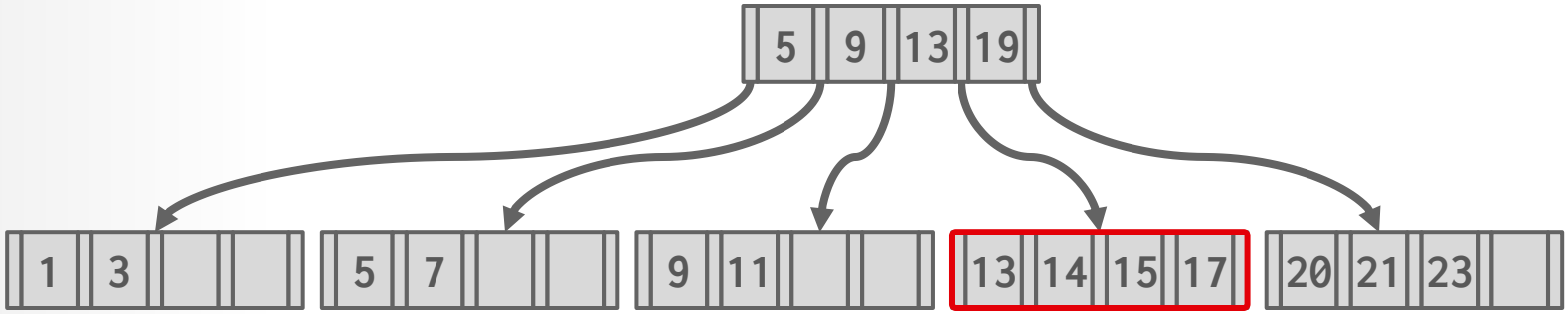


No space in the node where the new key "belongs".

B+TREE: INSERT EXAMPLE (3)

Insert 17

Insert 16

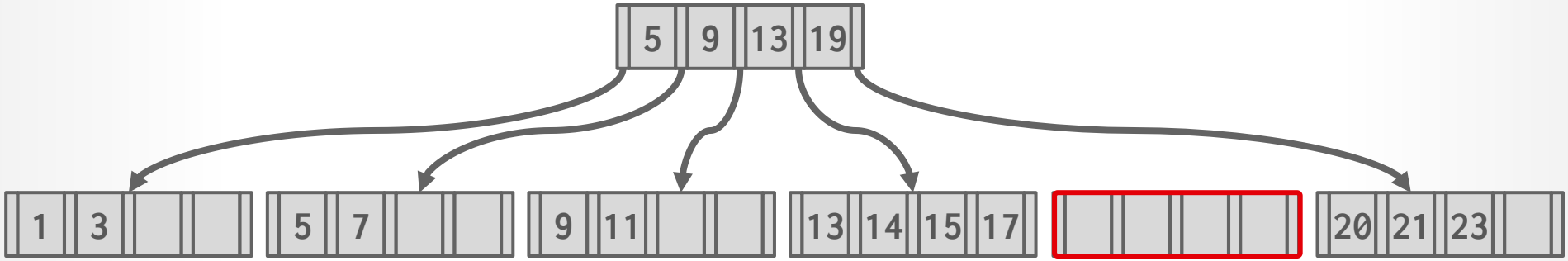


Split the node!
Copy the middle key.
Push the key up.

B+TREE: INSERT EXAMPLE (3)

Insert 17

Insert 16

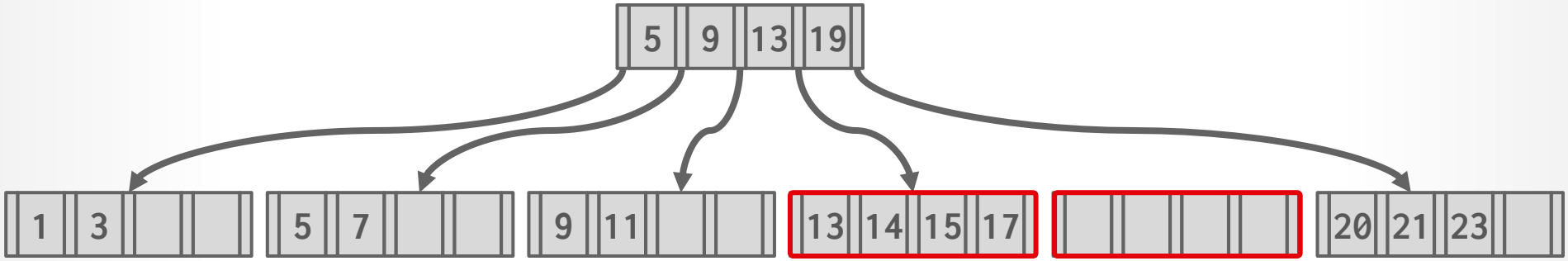


New Node!
Shuffle keys from the node that triggered the split.

B+TREE: INSERT EXAMPLE (3)

Insert 17

Insert 16

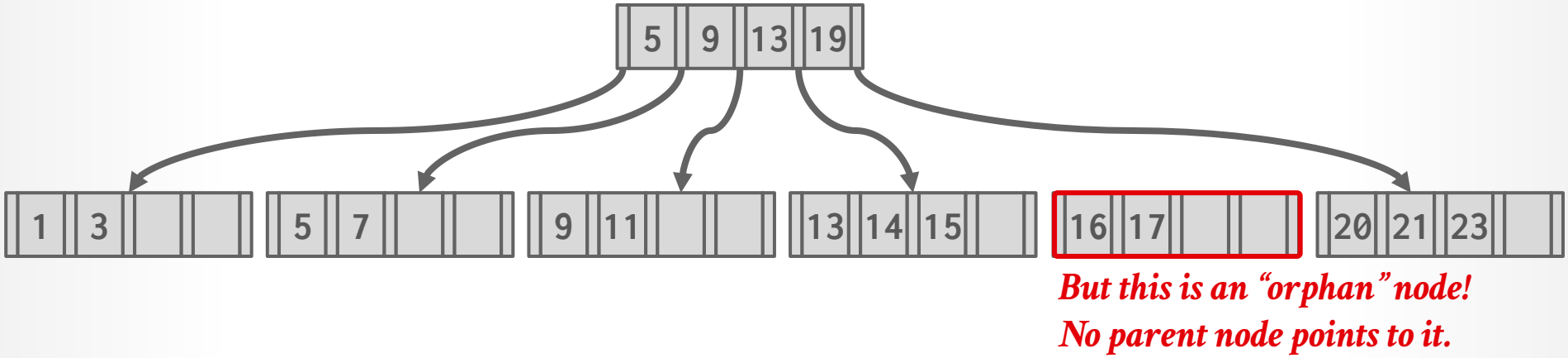


New Node!
Shuffle keys from the node that triggered the split.

B+TREE: INSERT EXAMPLE (3)

Insert 17

Insert 16



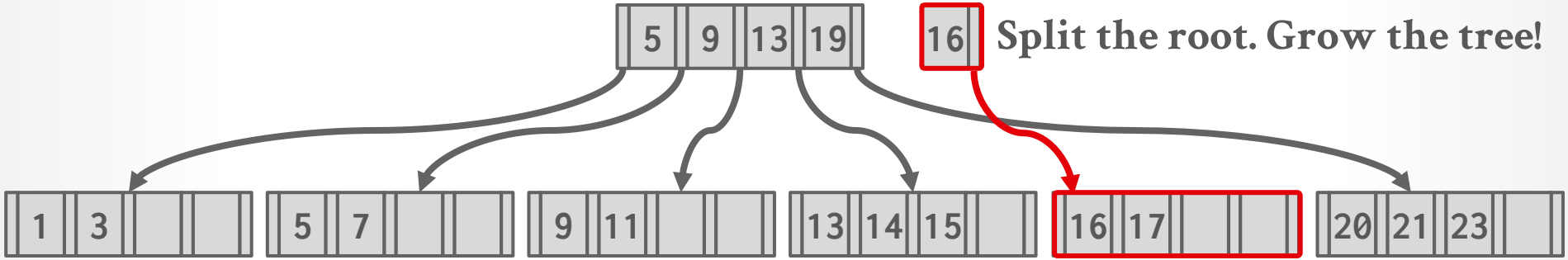
But this is an "orphan" node!
No parent node points to it.

B+TREE: INSERT EXAMPLE (3)

Insert 17

Insert 16

Want to create a key, pointer pair like this. But cannot insert it in the root node, which is full.



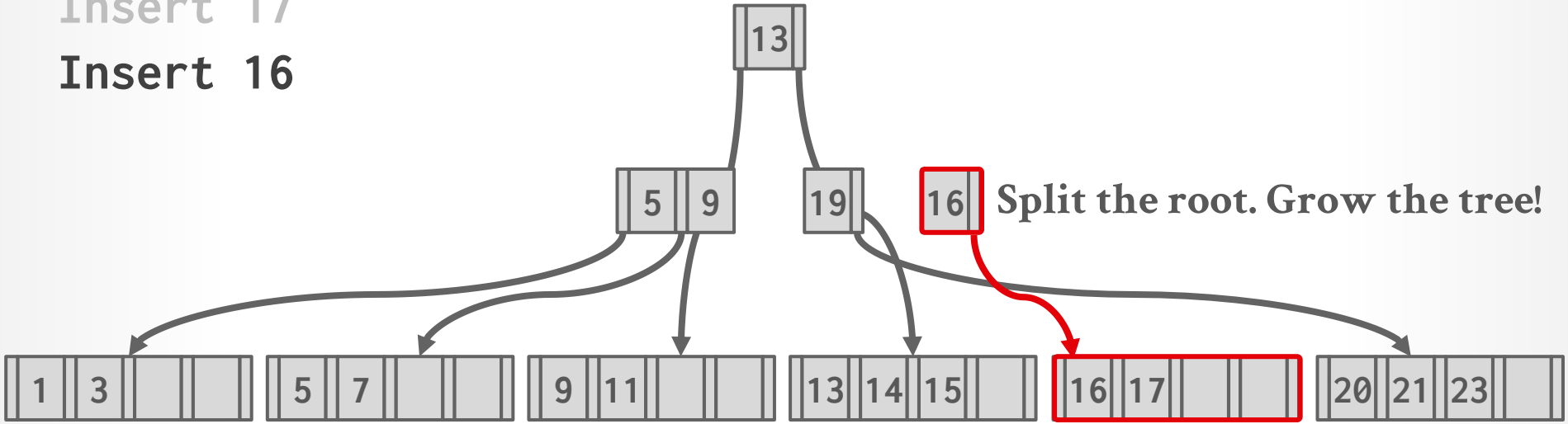
Split the root. Grow the tree!

But this is an "orphan" node! No parent node points to it.

B+TREE: INSERT EXAMPLE (3)

Insert 17

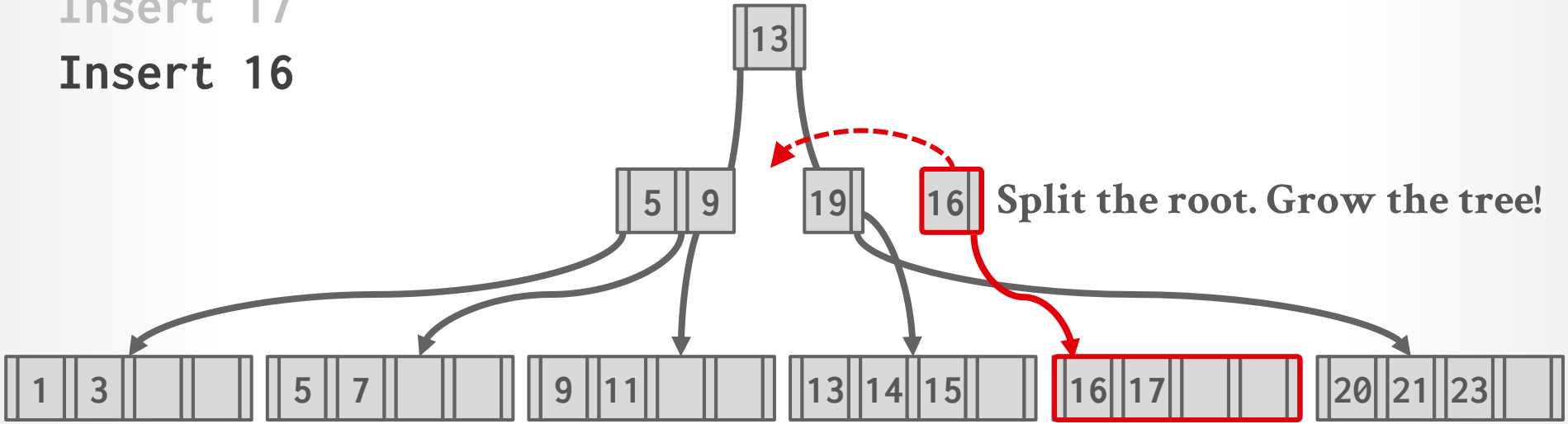
Insert 16



B+TREE: INSERT EXAMPLE (3)

Insert 17

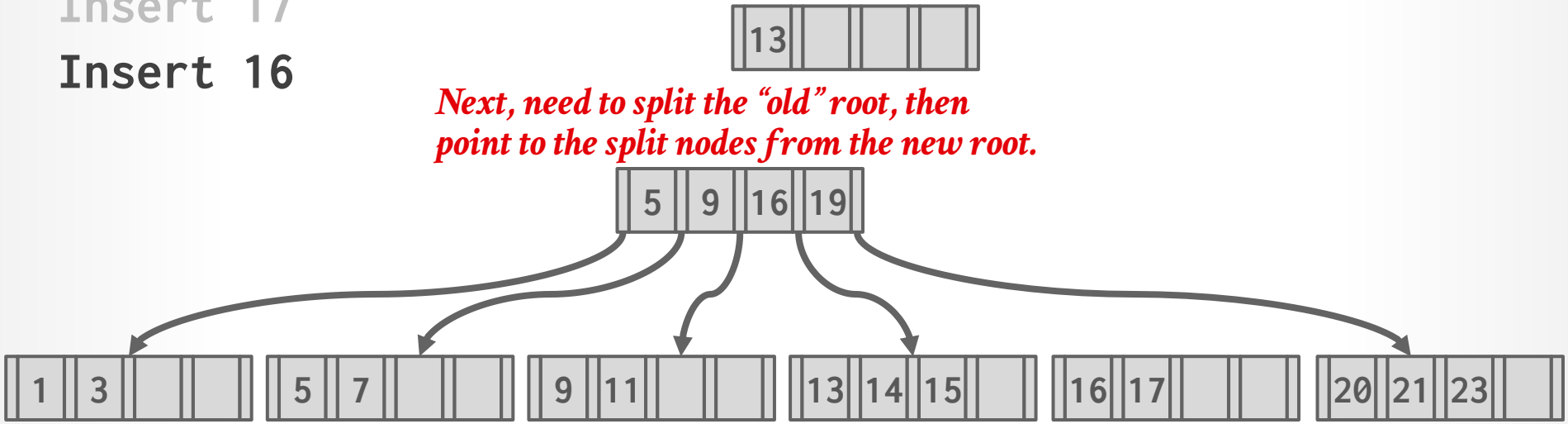
Insert 16



B+TREE: INSERT EXAMPLE (3)

Insert 17

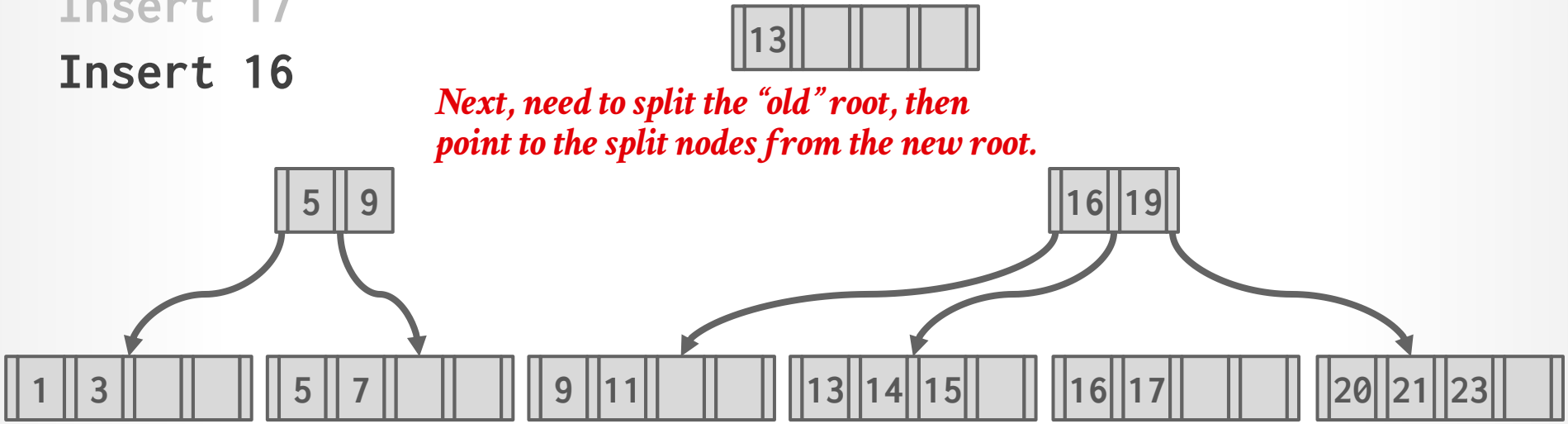
Insert 16



B+TREE: INSERT EXAMPLE (3)

Insert 17

Insert 16

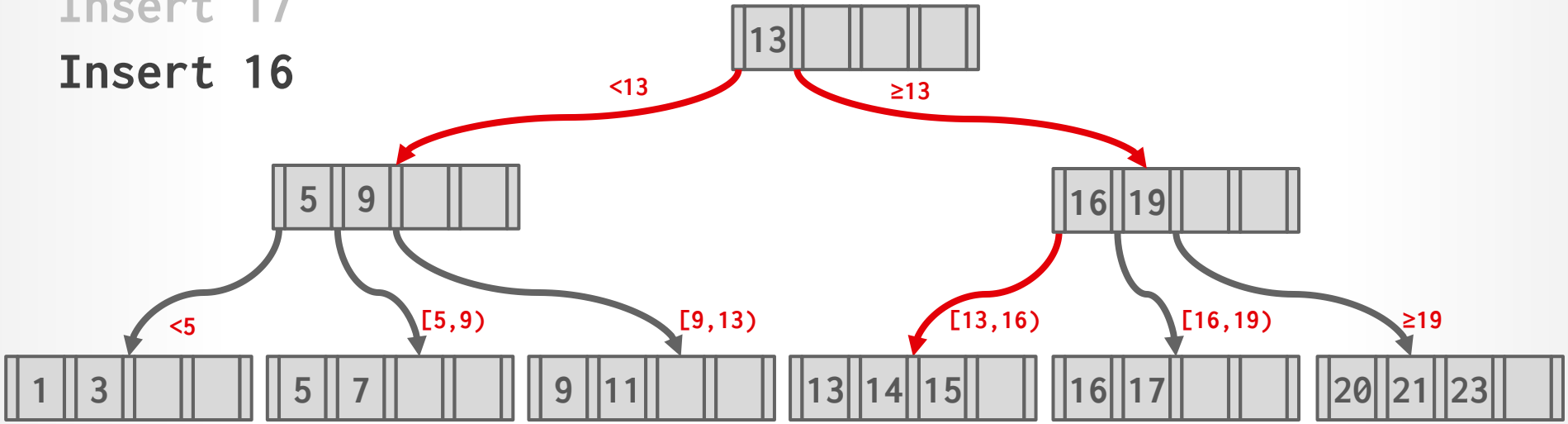


Next, need to split the "old" root, then point to the split nodes from the new root.

B+TREE: INSERT EXAMPLE (3)

Insert 17

Insert 16



B+TREE: DELETE

Start at root, find leaf **L** where entry belongs.
Remove the entry.
If **L** is at least half-full, done!
If **L** has only **m/2-1** entries,

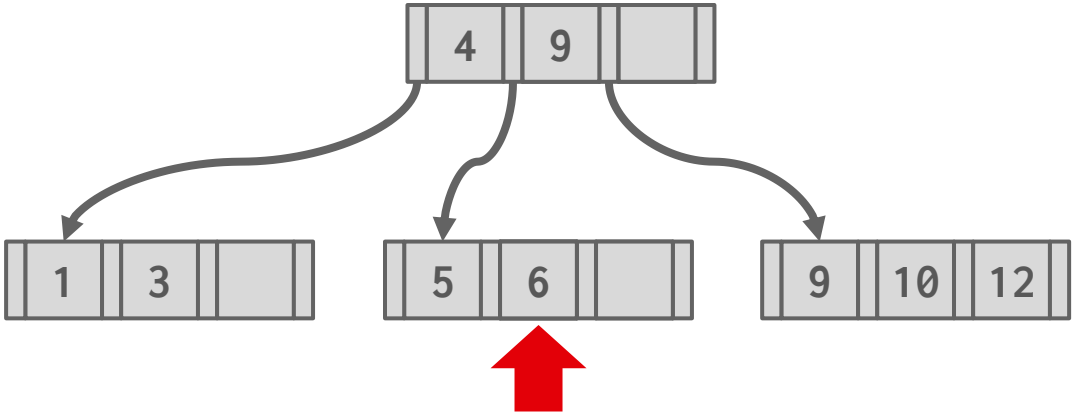
→ Try to re-distribute, borrowing from sibling (adjacent node with same parent as **L**).
→ If re-distribution fails, merge **L** and sibling.

If merge occurred, must delete entry (pointing to **L** or sibling) from parent of **L**.

Source: Chris Re

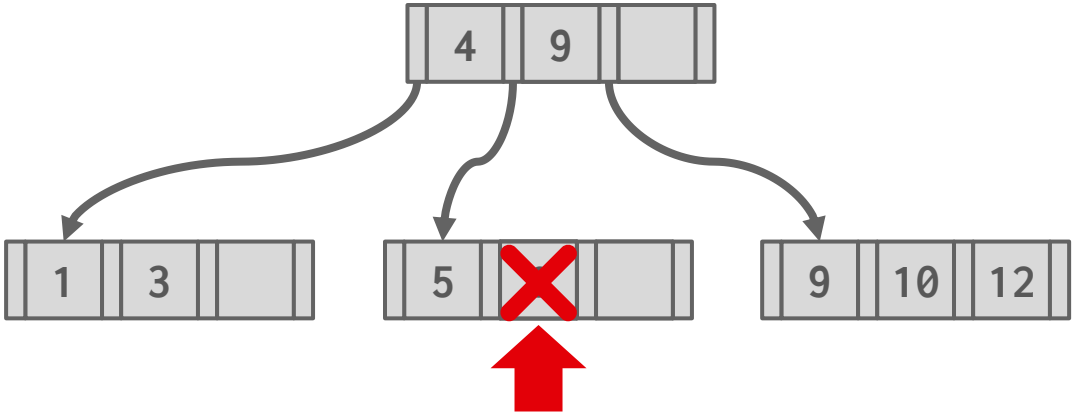
B+TREE: DELETE EXAMPLE (1)

Delete 6



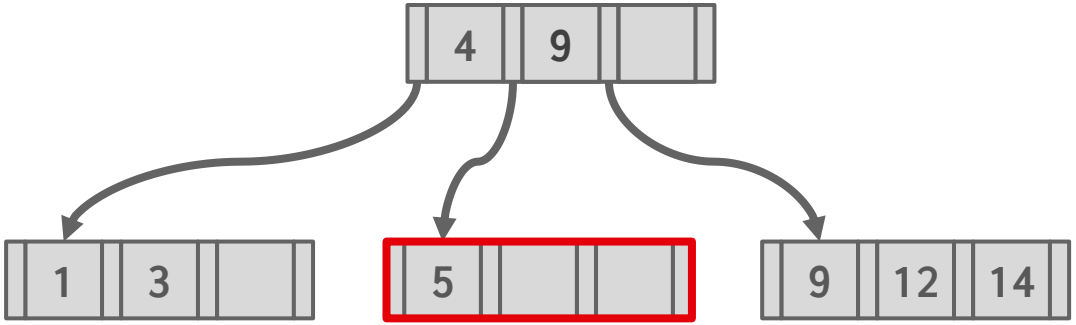
B+TREE: DELETE EXAMPLE (1)

Delete 6



B+TREE: DELETE EXAMPLE (1)

Delete 6

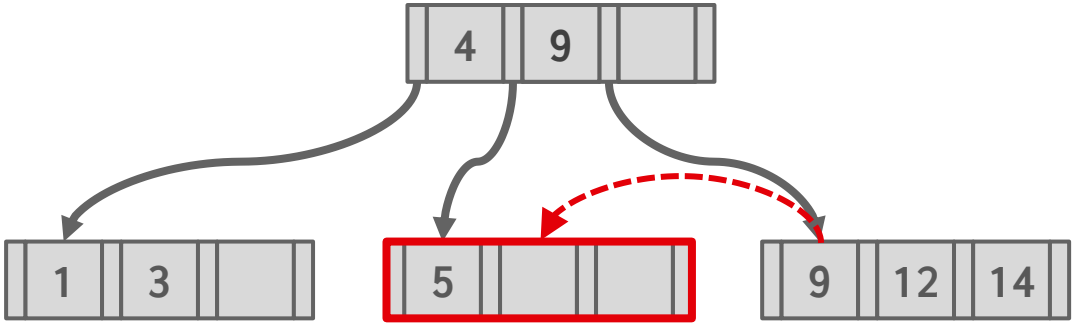


Borrow from a "rich" sibling node.

Could borrow from either sibling.

B+TREE: DELETE EXAMPLE (1)

Delete 6

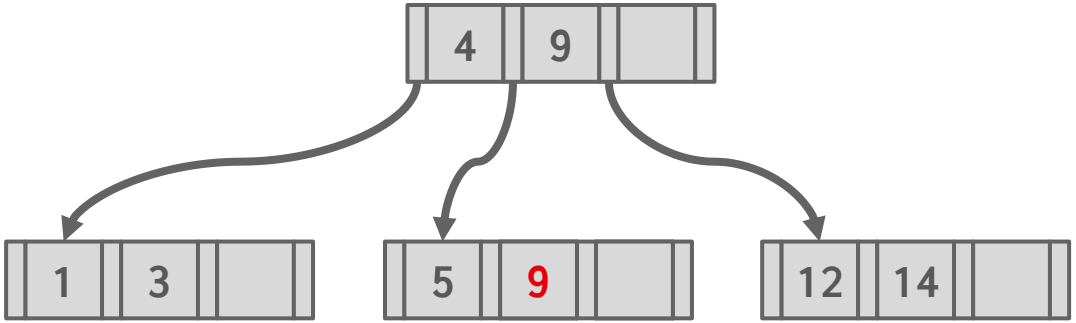


Borrow from a "rich" sibling node.

Could borrow from either sibling.

B+TREE: DELETE EXAMPLE (1)

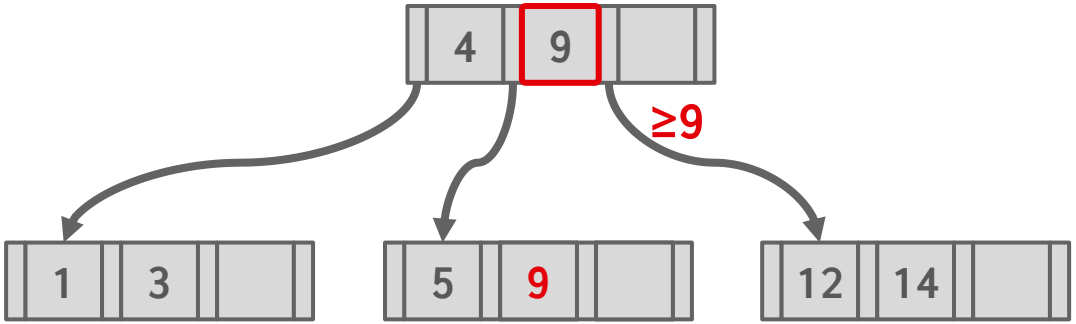
Delete 6



Need to update parent node!

B+TREE: DELETE EXAMPLE (1)

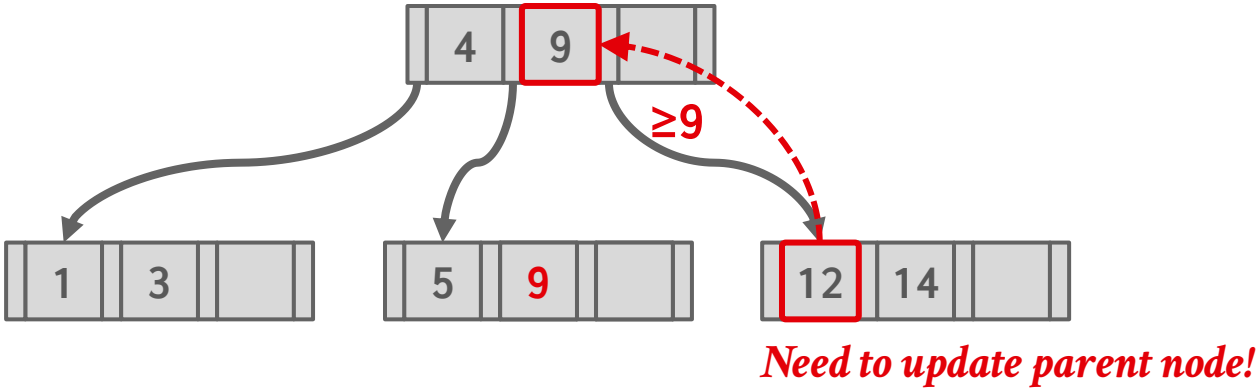
Delete 6



Need to update parent node!

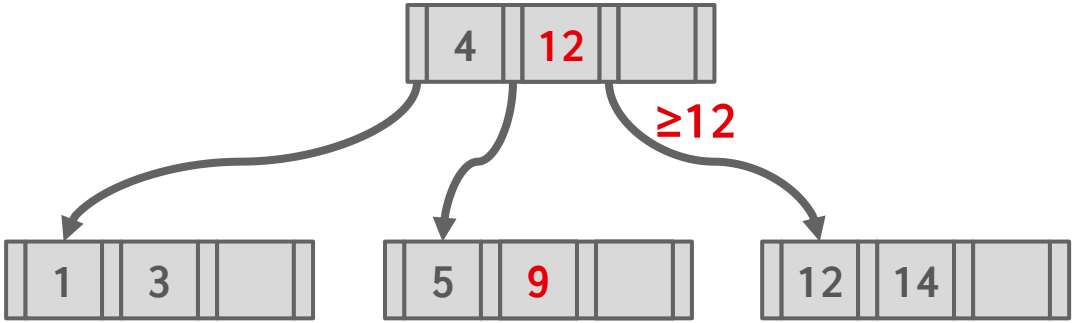
B+TREE: DELETE EXAMPLE (1)

Delete 6



B+TREE: DELETE EXAMPLE (1)

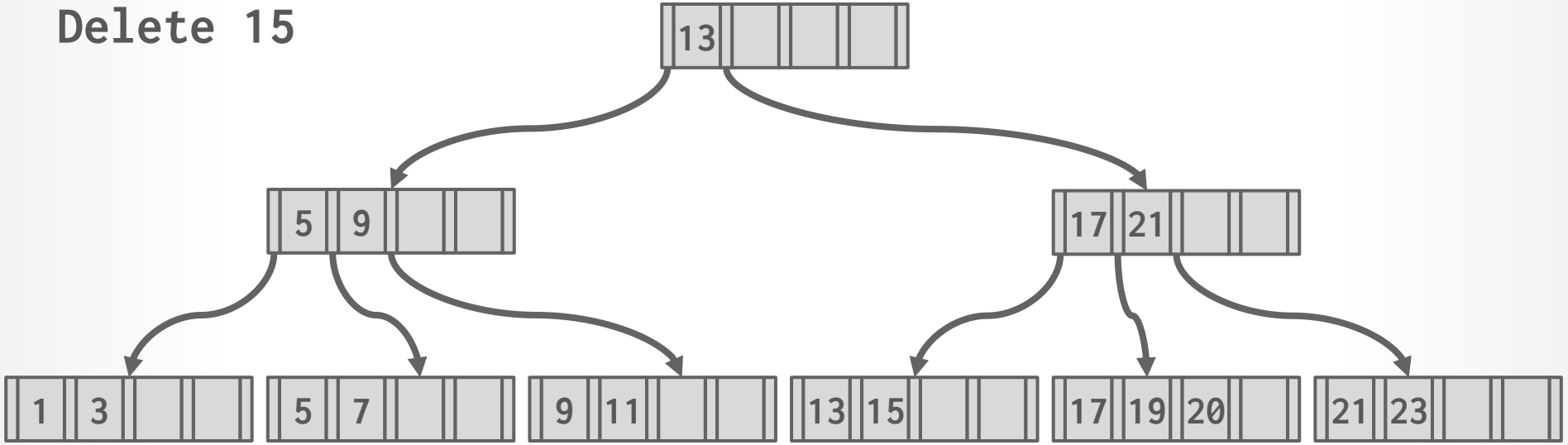
Delete 6



Need to update parent node!

B+TREE: DELETE EXAMPLE (2)

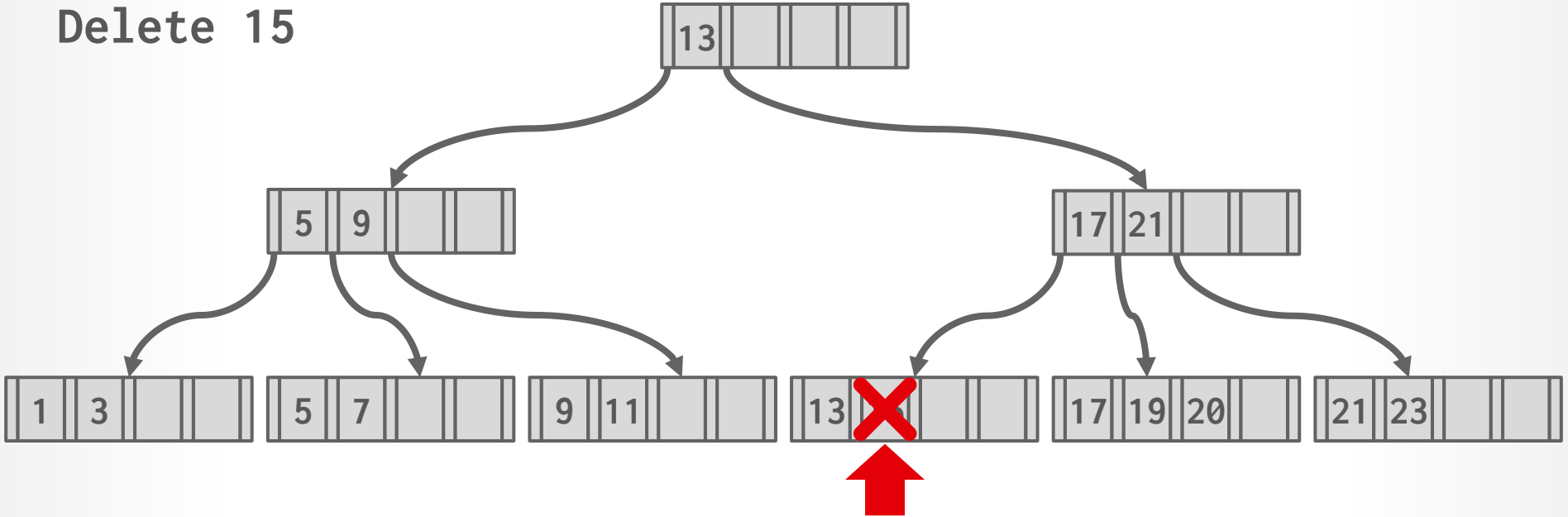
Delete 15



Note: New Example/Tree.

B+TREE: DELETE EXAMPLE (2)

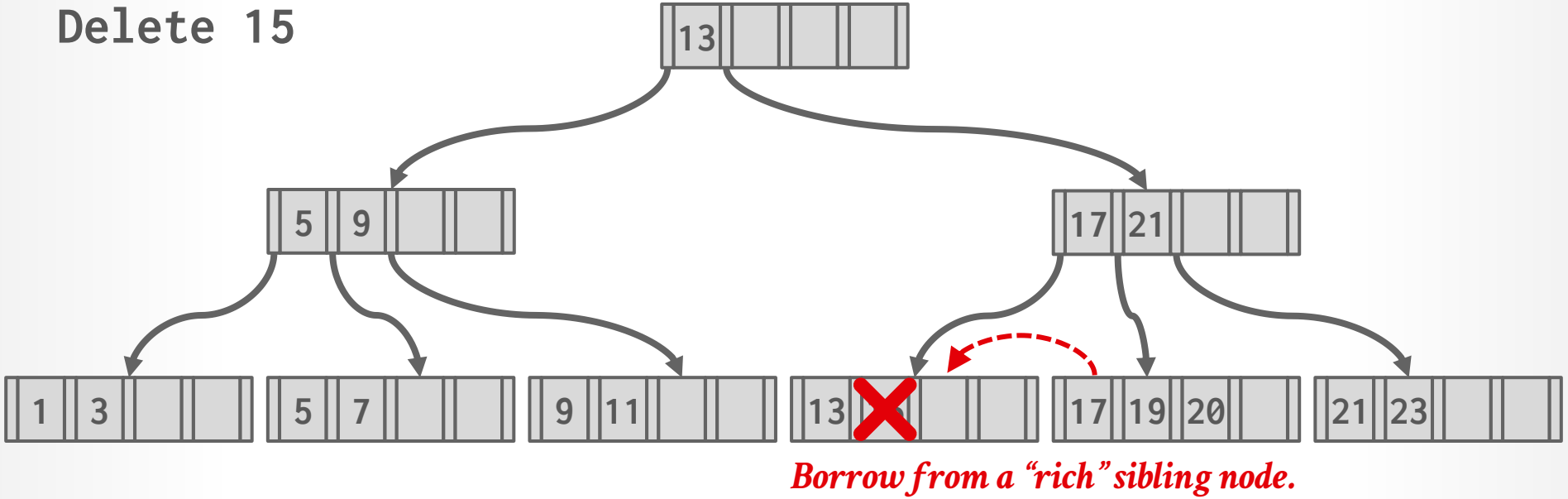
Delete 15



Note: New Example/Tree.

B+TREE: DELETE EXAMPLE (2)

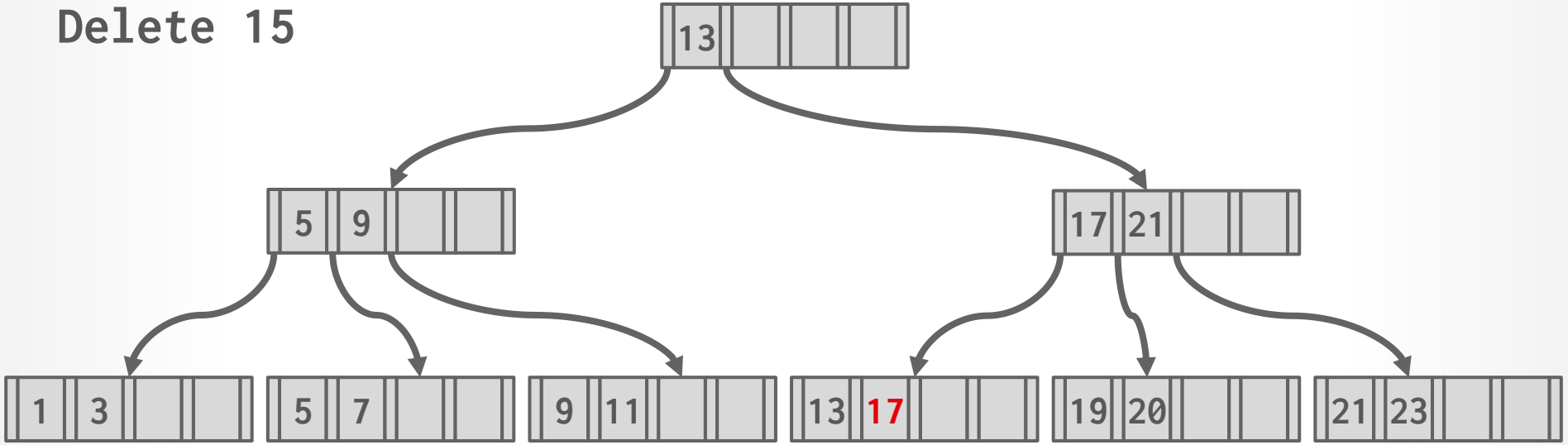
Delete 15



Note: New Example/Tree.

B+TREE: DELETE EXAMPLE (2)

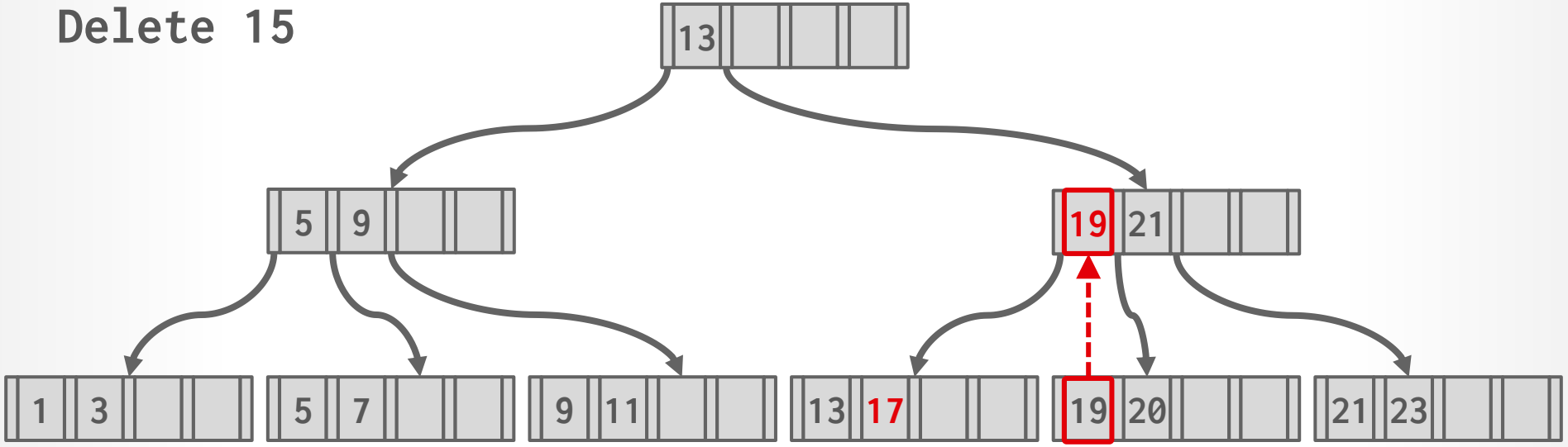
Delete 15



Need to update parent node!

B+TREE: DELETE EXAMPLE (2)

Delete 15

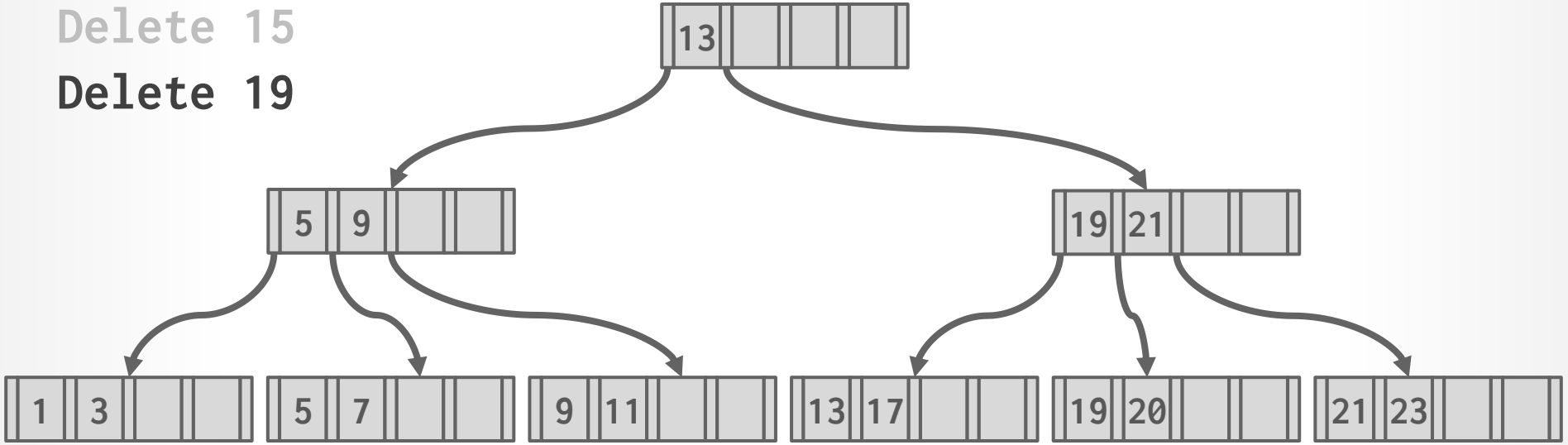


Need to update parent node!

B+TREE: DELETE EXAMPLE (3)

Delete 15

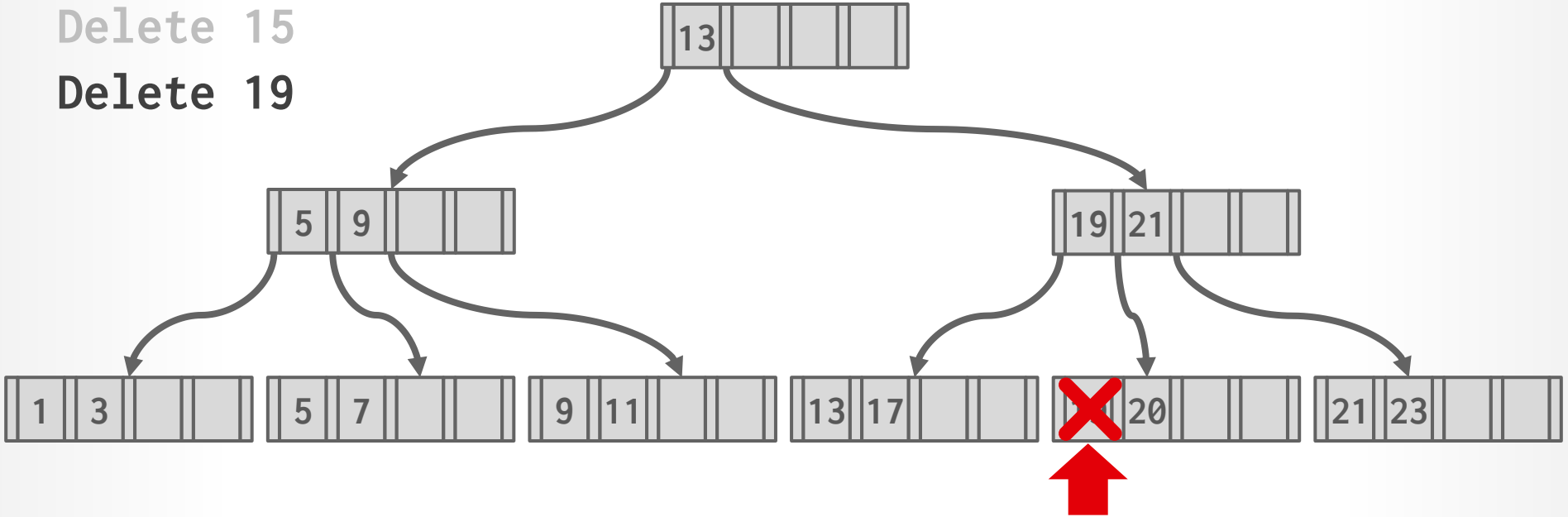
Delete 19



B+TREE: DELETE EXAMPLE (3)

Delete 15

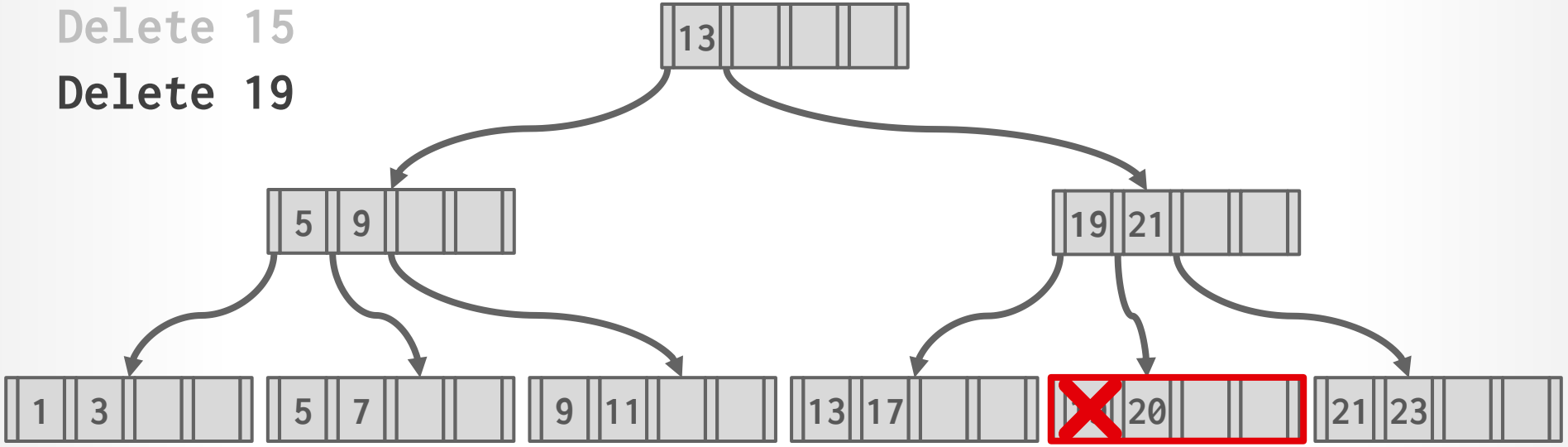
Delete 19



B+TREE: DELETE EXAMPLE (3)

Delete 15

Delete 19

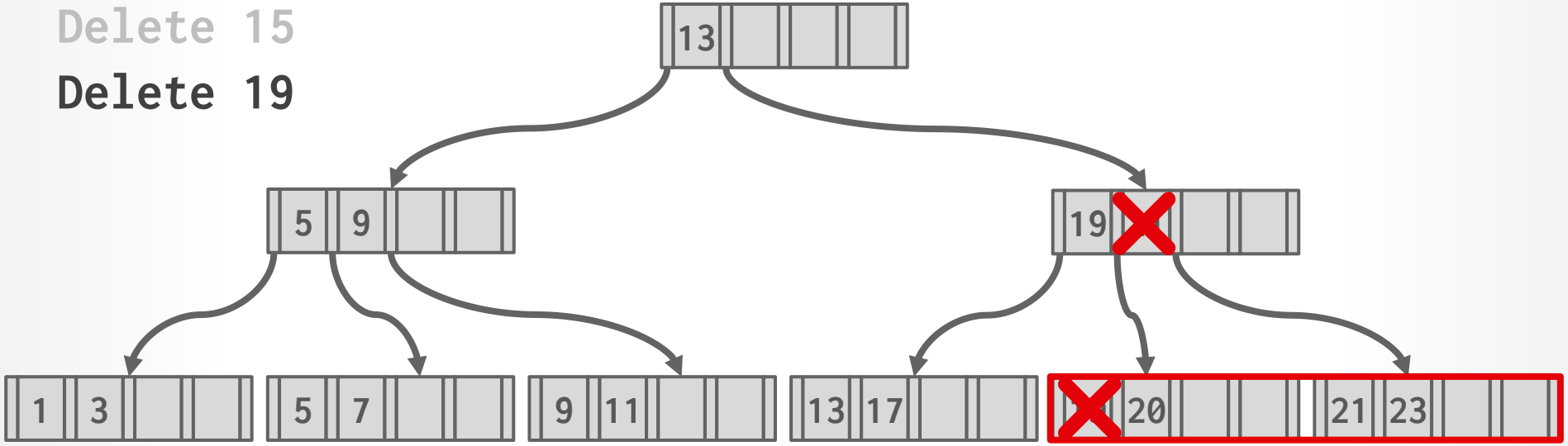


Under-filled!
No "rich" sibling nodes to borrow.
Merge with a sibling

B+TREE: DELETE EXAMPLE (3)

Delete 15

Delete 19

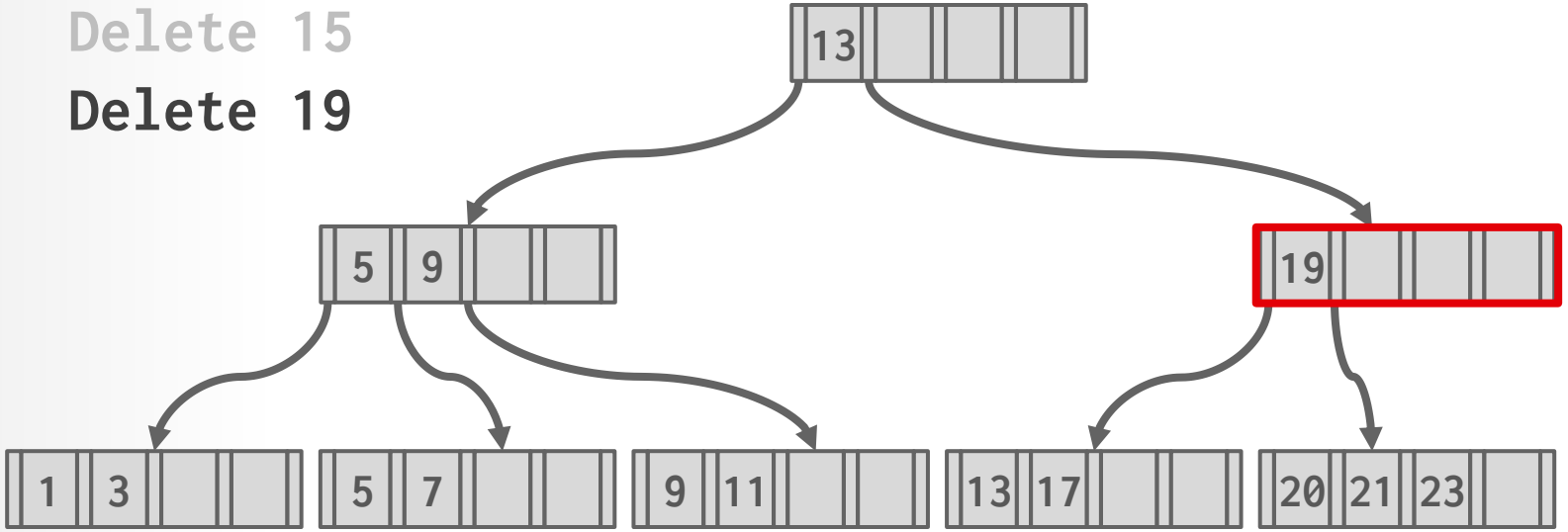


Under-filled!
No "rich" sibling nodes to borrow.
Merge with a sibling

B+TREE: DELETE EXAMPLE (3)

Delete 15

Delete 19

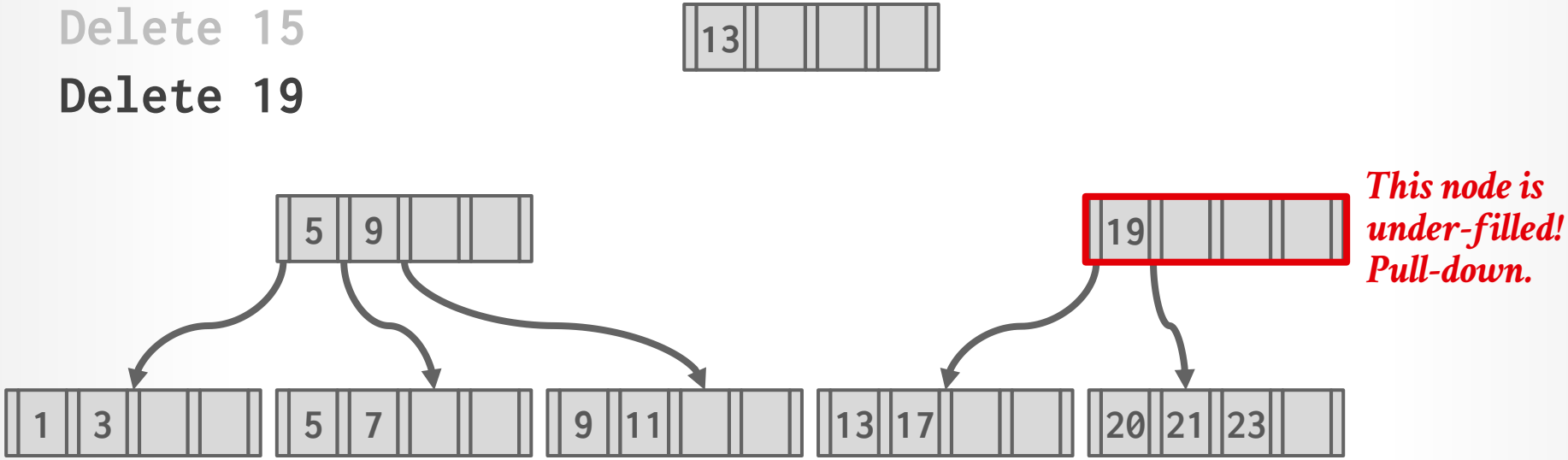


This node is under-filled! Pull-down.

B+TREE: DELETE EXAMPLE (3)

Delete 15

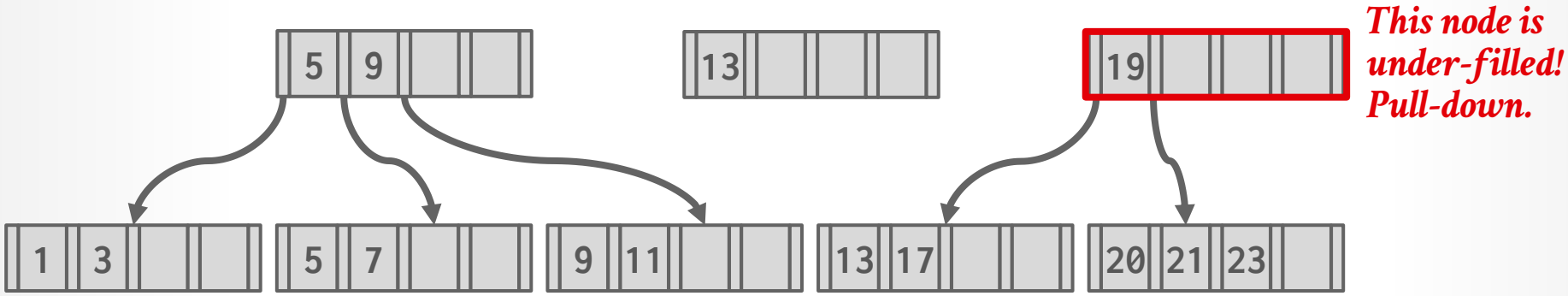
Delete 19



B+TREE: DELETE EXAMPLE (3)

Delete 15

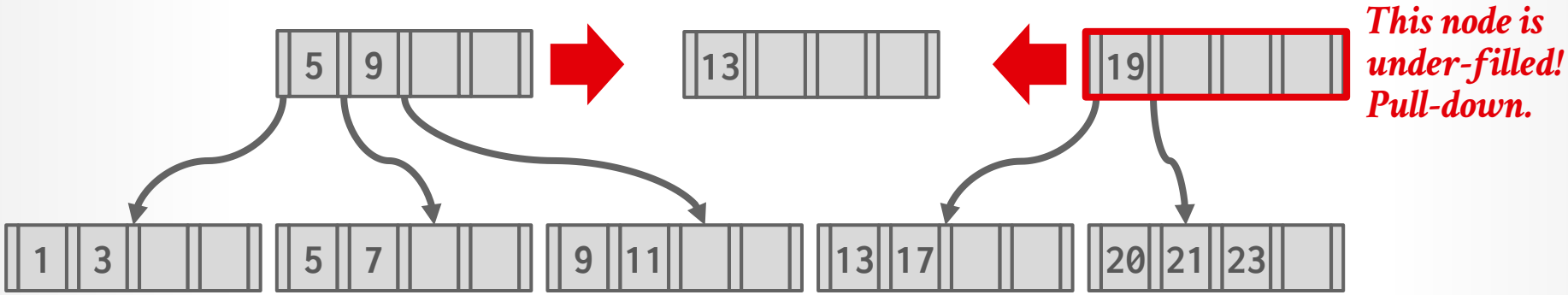
Delete 19



B+TREE: DELETE EXAMPLE (3)

Delete 15

Delete 19

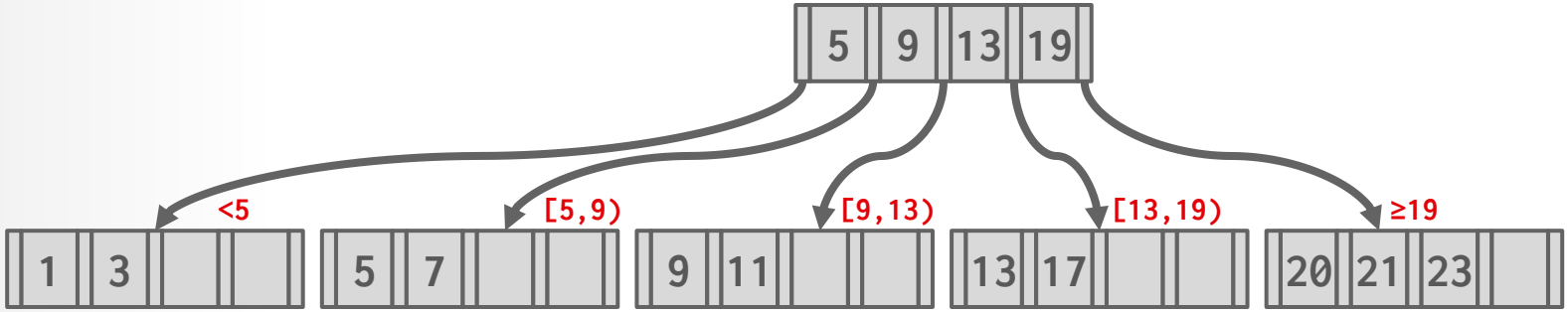


B+TREE: DELETE EXAMPLE (3)

Delete 15

Delete 19

The tree has shrunk in height.



COMPOSITE INDEX

A **composite index** is when the key is comprised of two or more attributes.

→ Example: Index on **<a,b,c>**

```
CREATE INDEX my_idx ON xxx (a, b DESC, c NULLS FIRST);
```

DBMS can use B+Tree index if the query provides a "prefix" of composite key.

→ Supported: **(a=1 AND b=2 AND c=3)**

→ Supported: **(a=1 AND b=2)**

→ Rarely Supported: **(b=2)**, **(c=3)**

COMPOSITE INDEX

A **composite index** is when the key is comprised of two or more attributes.

→ Example: Index on **<a,b,c>**

Sort Order

```
CREATE INDEX my_idx ON xxx (a, b DESC, c NULLS FIRST);
```

Null Handling

DBMS can use B+Tree index if the query provides a "prefix" of composite key.

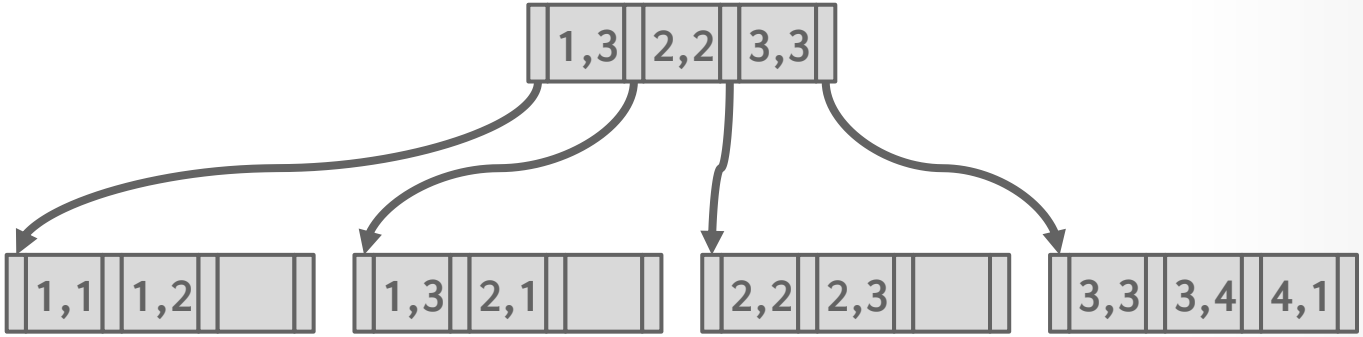
→ Supported: **(a=1 AND b=2 AND c=3)**

→ Supported: **(a=1 AND b=2)**

→ Rarely Supported: **(b=2)**, **(c=3)**

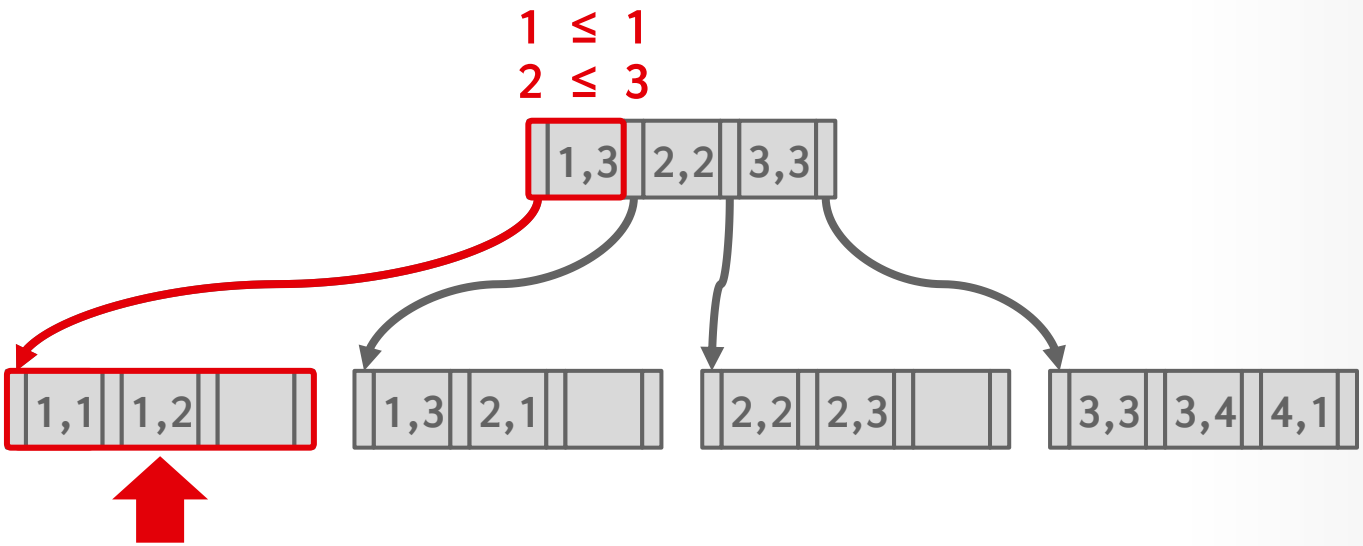
SELECTION CONDITIONS

Find Key=(1,2)



SELECTION CONDITIONS

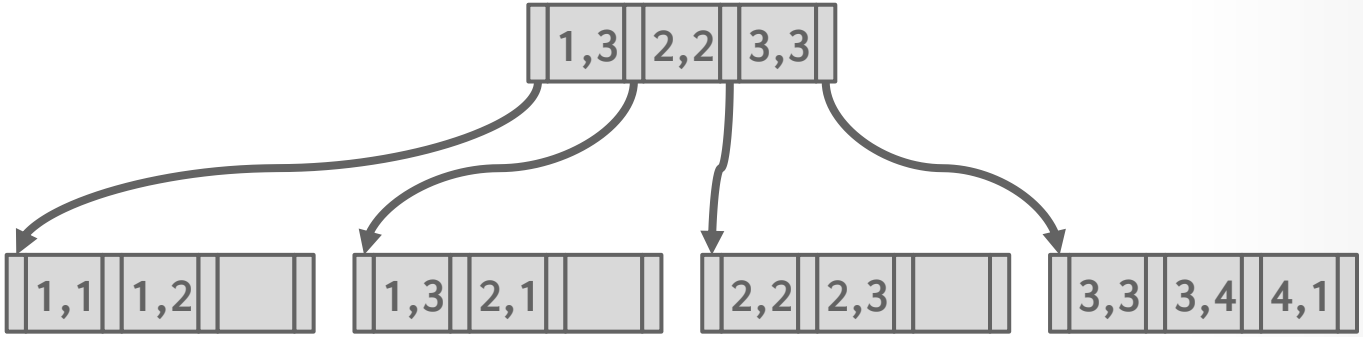
Find Key=(1,2)



SELECTION CONDITIONS

Find Key=(1,2)

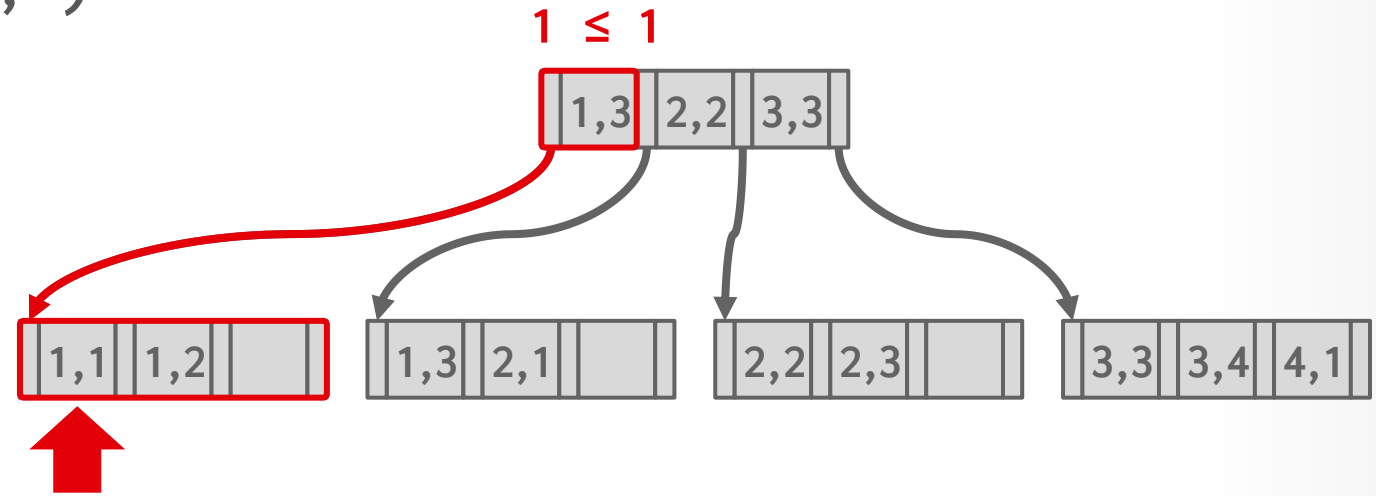
Find Key=(1,*)



SELECTION CONDITIONS

Find Key=(1,2)

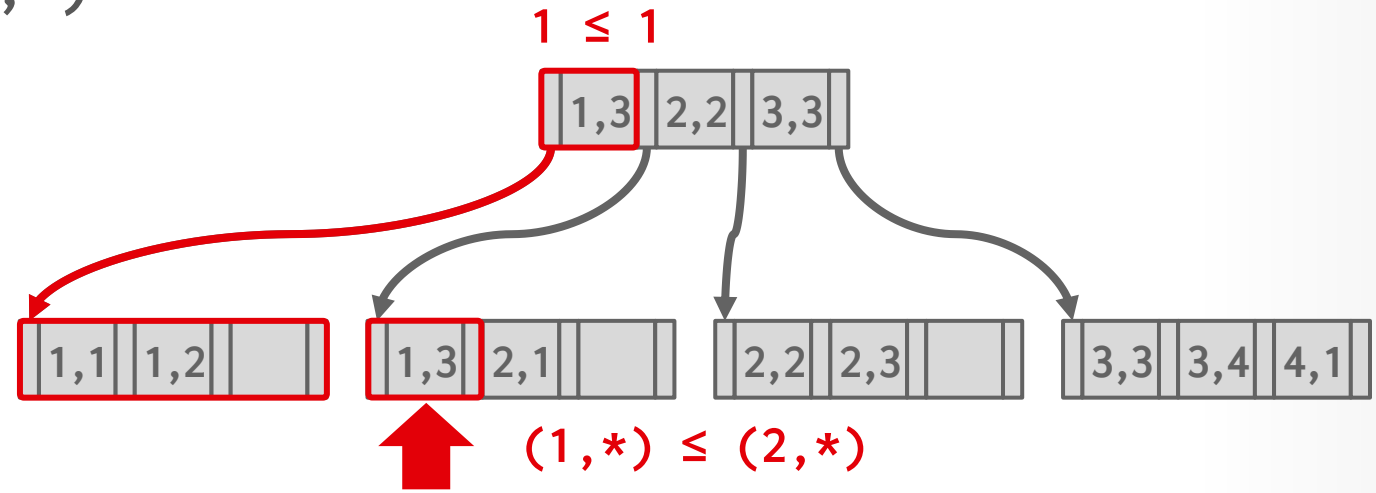
Find Key=(1,*)



SELECTION CONDITIONS

Find Key=(1,2)

Find Key=(1,*)

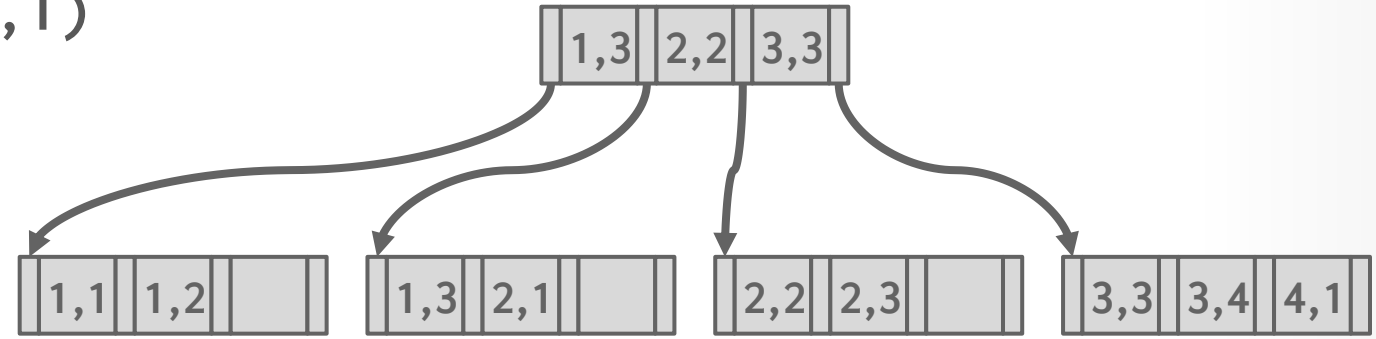


SELECTION CONDITIONS

Find Key=(1,2)

Find Key=(1,*)

Find Key=(*,1)

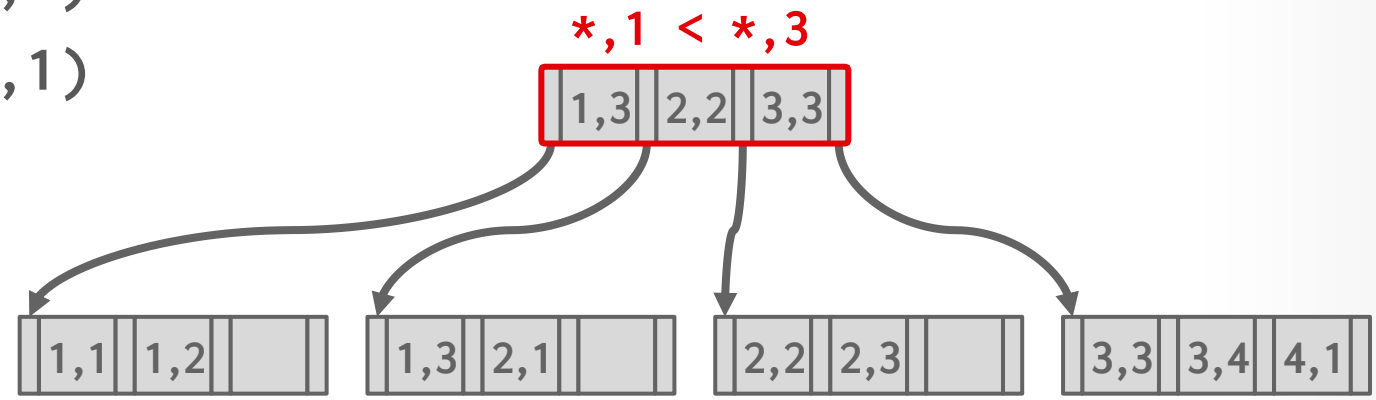


SELECTION CONDITIONS

Find Key=(1,2)

Find Key=(1,*)

Find Key=(*,1)

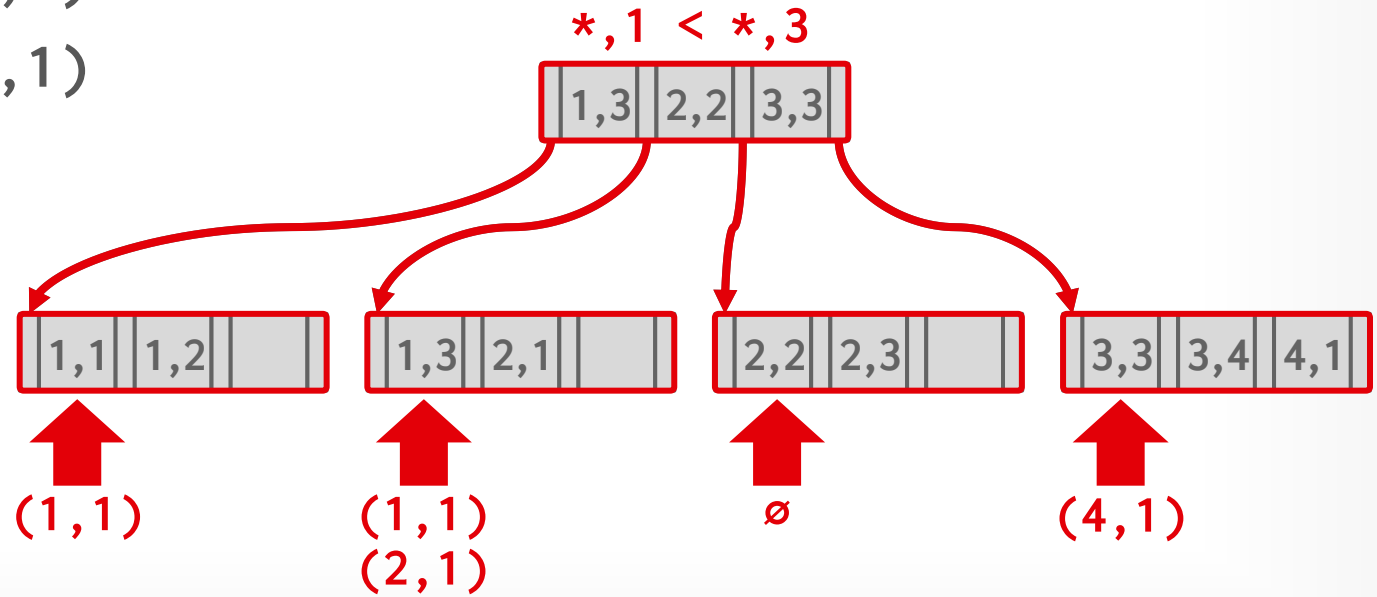


SELECTION CONDITIONS

Find Key=(1,2)

Find Key=(1,*)

Find Key=(*,1)



B+TREE: DUPLICATE KEYS

Approach #1: Append Record ID

→ Add the tuple's unique Record ID as part of the key to ensure that all keys are unique.

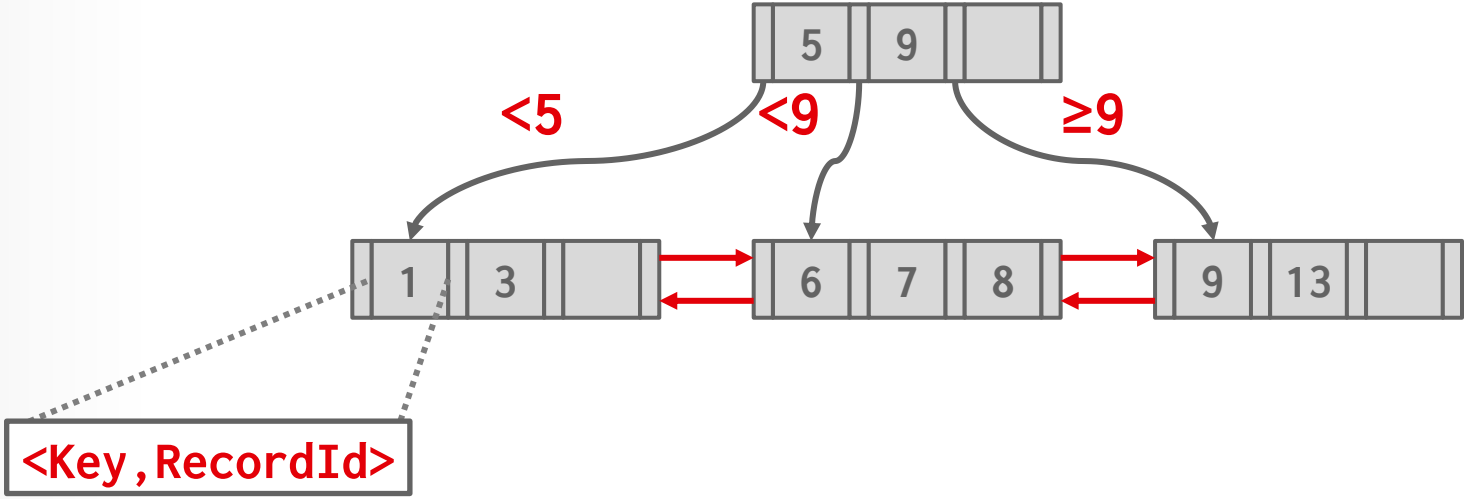
→ The DBMS can still use partial keys to find tuples.

Approach #2: Overflow Leaf Nodes

→ Allow leaf nodes to spill into overflow nodes that contain the duplicate keys.

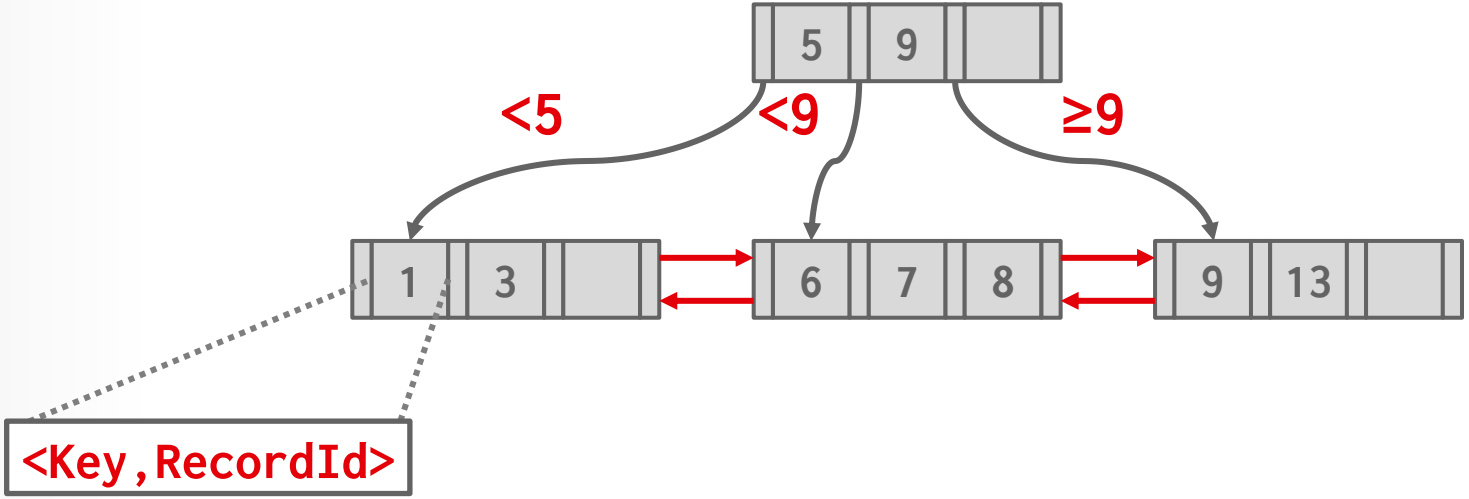
→ This is more complex to maintain and modify.

B+TREE: APPEND RECORD ID



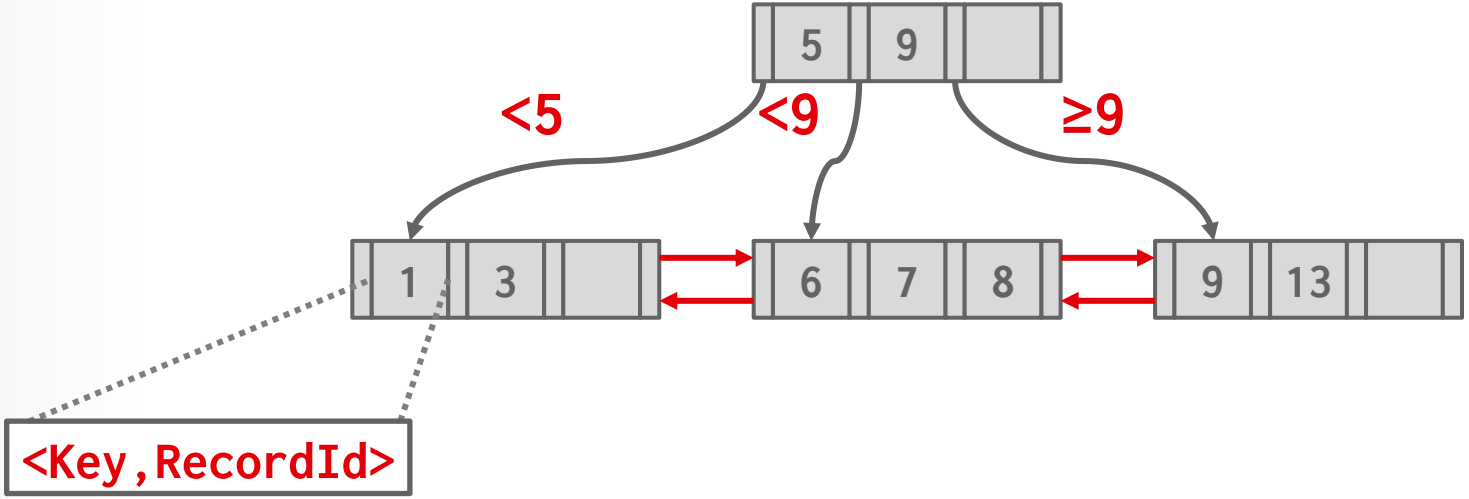
B+TREE: APPEND RECORD ID

Insert 6



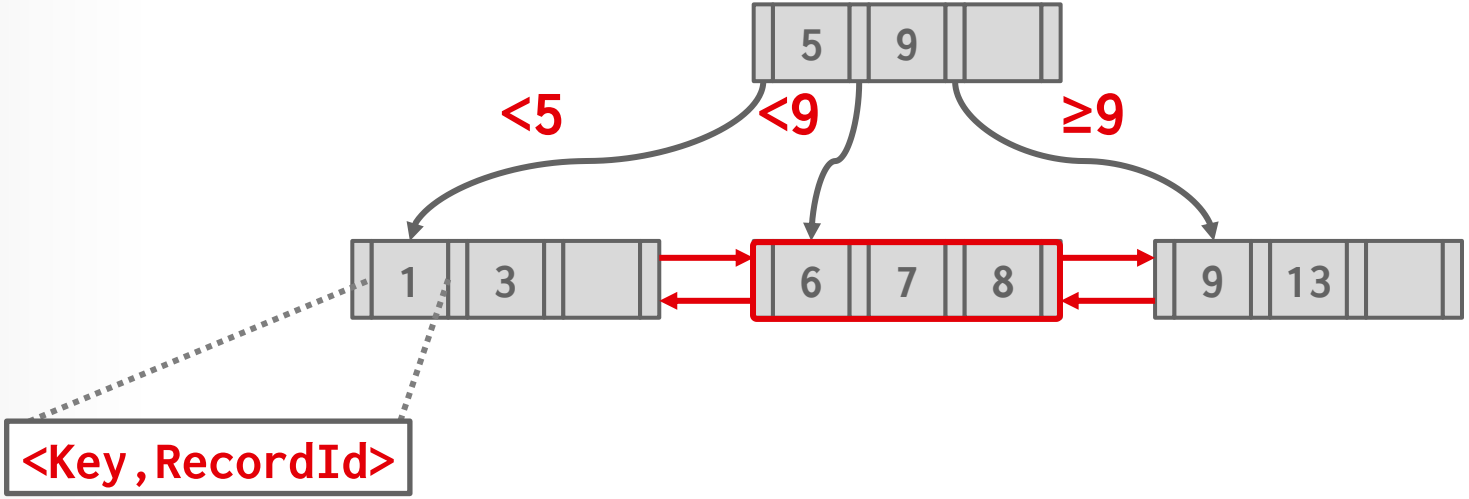
B+TREE: APPEND RECORD ID

Insert <6,(Page,Slot)>



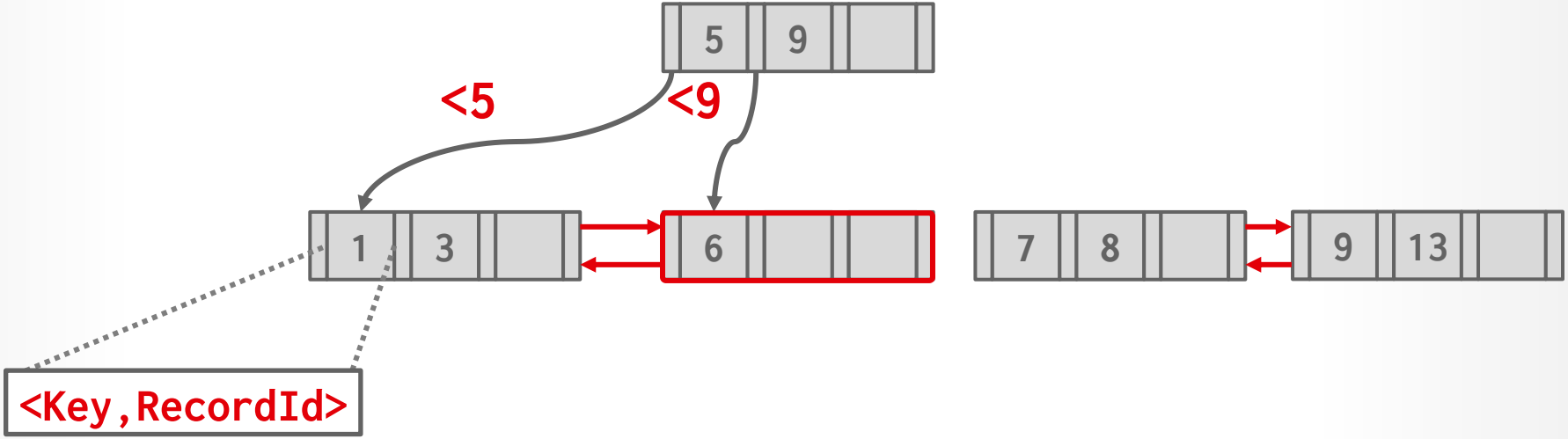
B+TREE: APPEND RECORD ID

Insert <6,(Page,Slot)>



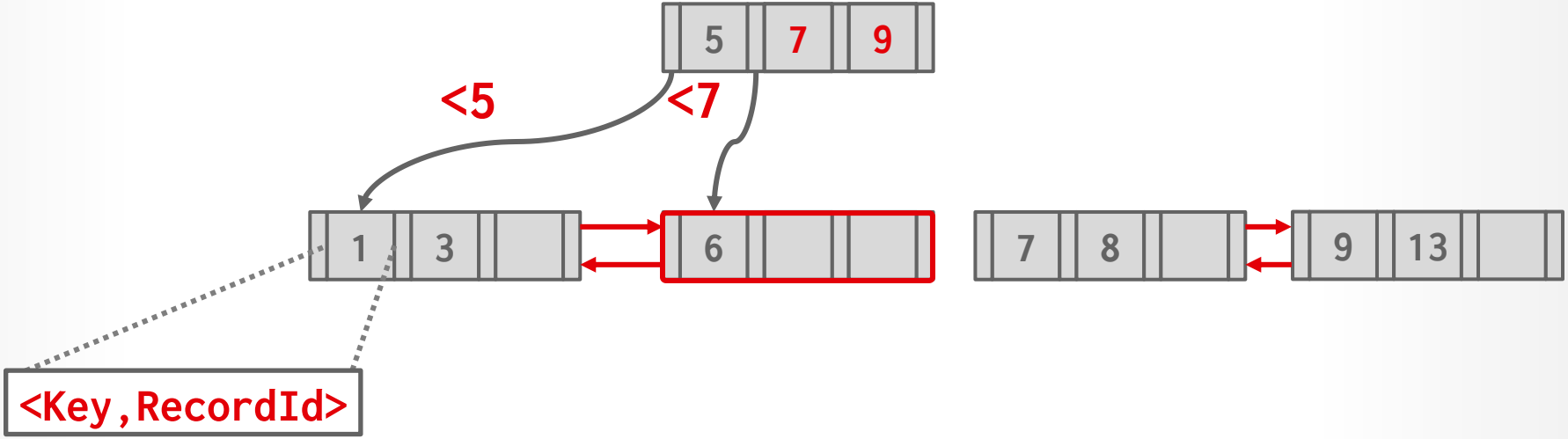
B+TREE: APPEND RECORD ID

Insert <6,(Page,Slot)>



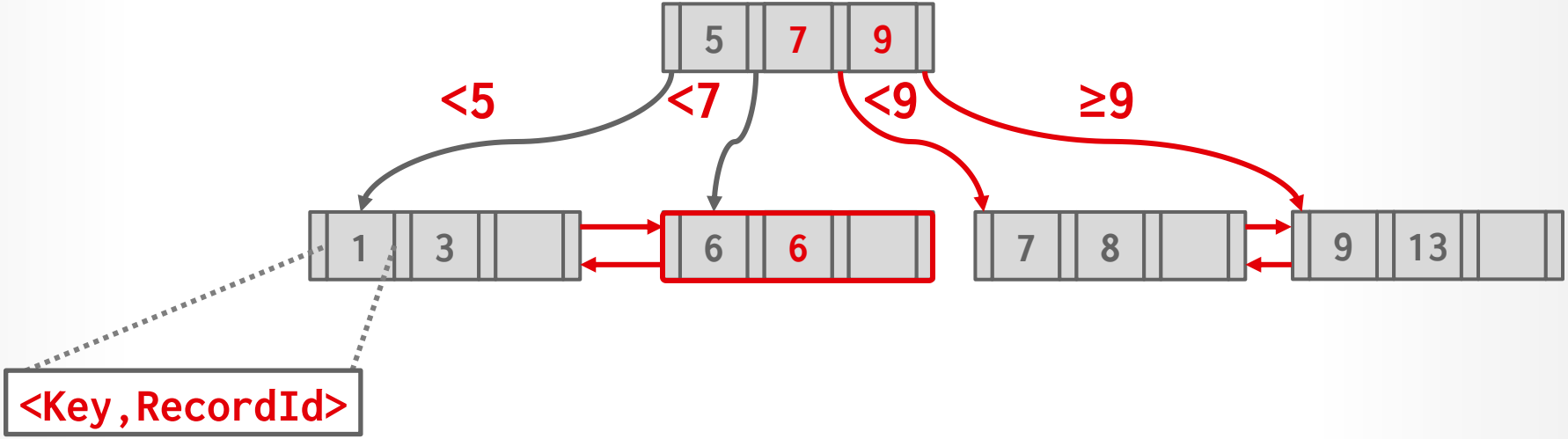
B+TREE: APPEND RECORD ID

Insert <6,(Page,Slot)>



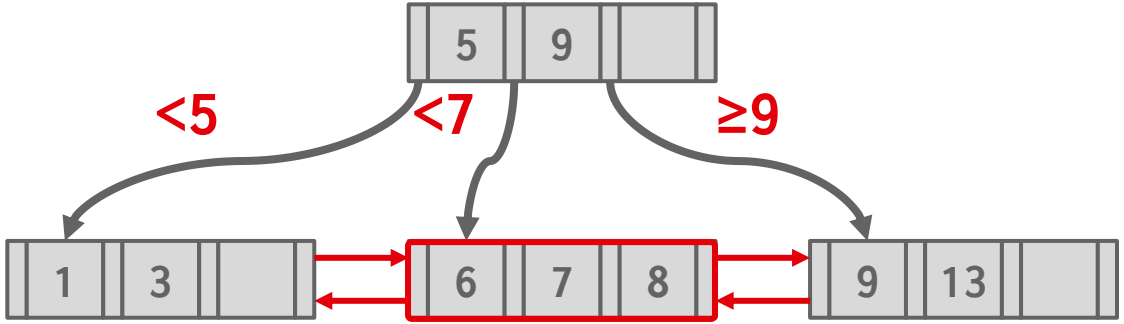
B+TREE: APPEND RECORD ID

Insert <6,(Page,Slot)>



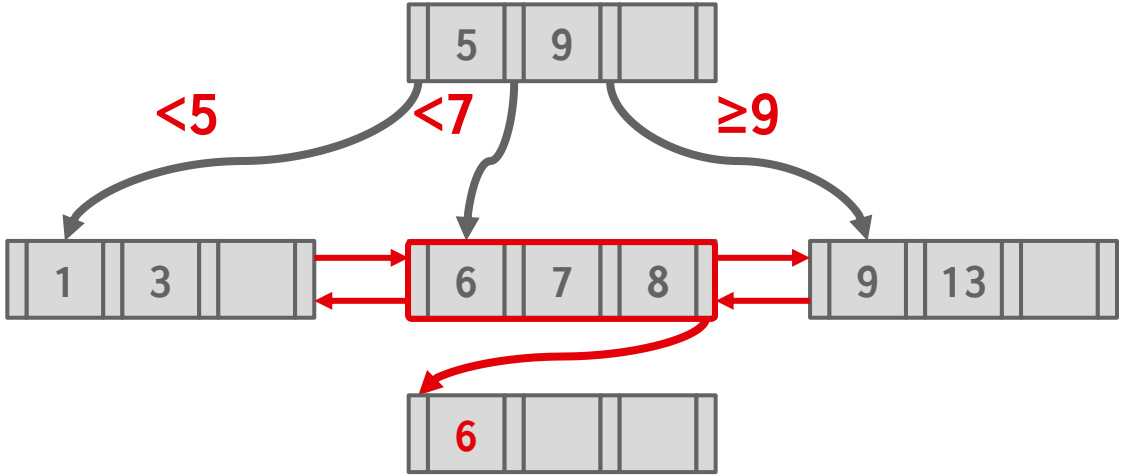
B+TREE: OVERFLOW LEAF NODES

Insert 6



B+TREE: OVERFLOW LEAF NODES

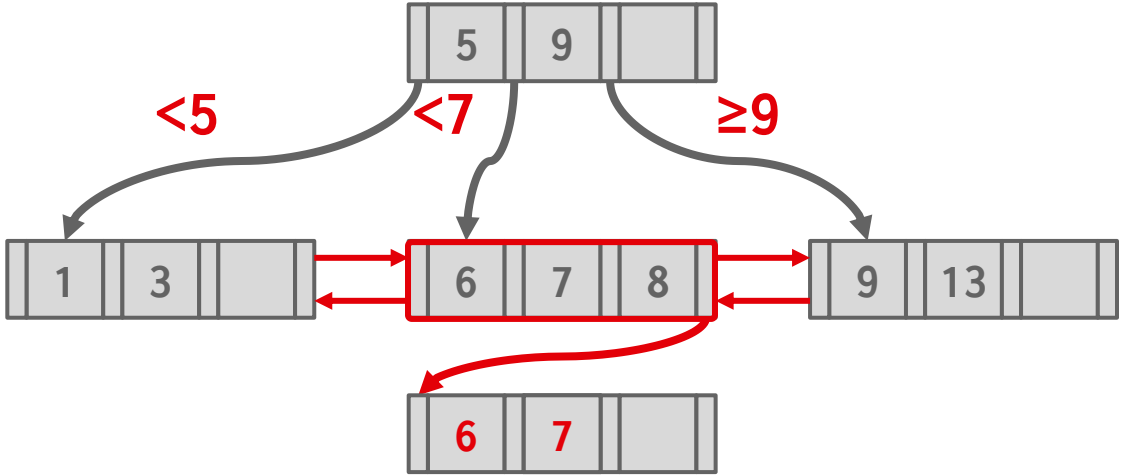
Insert 6



B+TREE: OVERFLOW LEAF NODES

Insert 6

Insert 7

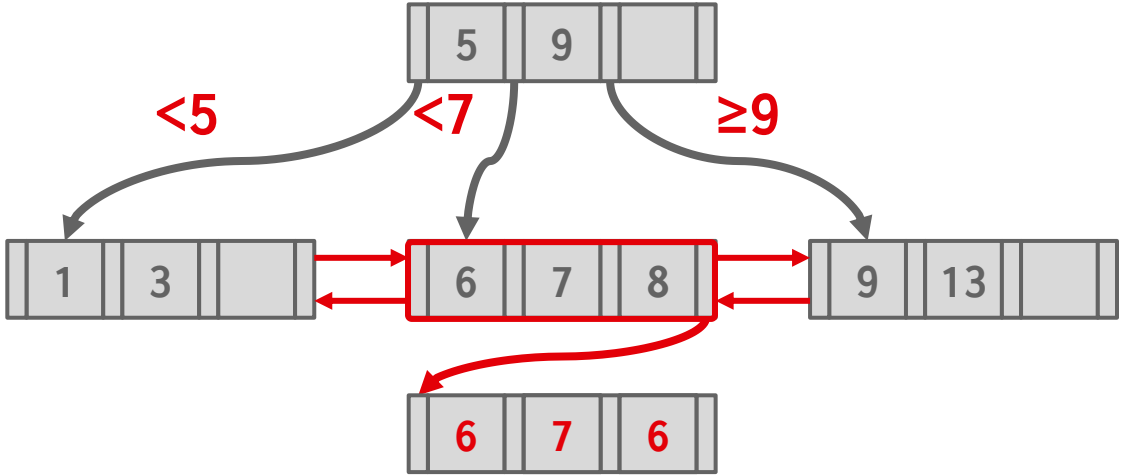


B+TREE: OVERFLOW LEAF NODES

Insert 6

Insert 7

Insert 6



CLUSTERED INDEXES

The table is stored in the sort order specified by the primary key.

→ Can be either heap- or index-organized storage.

Some DBMSs always use a clustered index.

→ If a table does not contain a primary key, the DBMS will automatically make a hidden primary key.

Other DBMSs cannot use them at all.

CLUSTERED B+TREE

Traverse to the left-most leaf page and then retrieve tuples from all leaf pages.

This will always be better than sorting data for each query.

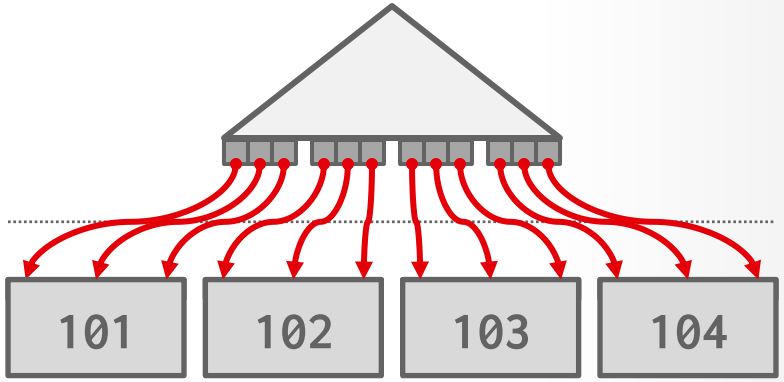
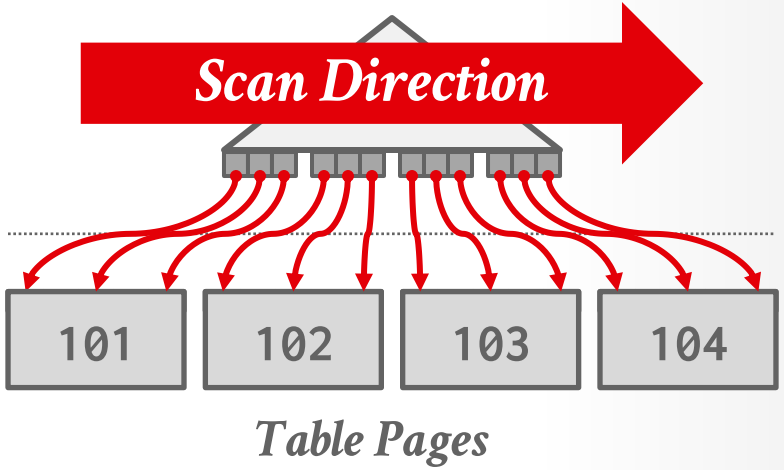


Table Pages

CLUSTERED B+TREE

Traverse to the left-most leaf page and then retrieve tuples from all leaf pages.

This will always be better than sorting data for each query.

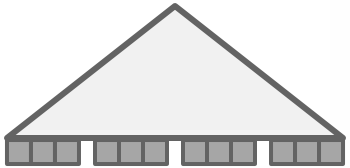


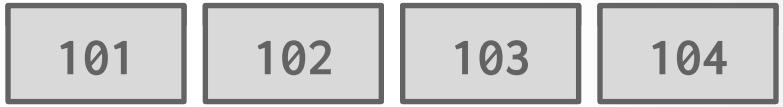
INDEX SCAN PAGE SORTING

Retrieving tuples in the order they appear in a non-clustered index is inefficient due to redundant reads.

A better approach is to find all the tuples that the query needs and then sort them based on their page ID.

The DBMS retrieves each page once.



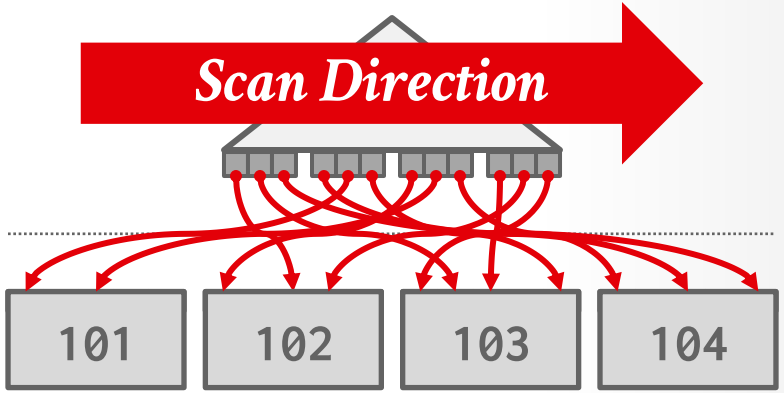


INDEX SCAN PAGE SORTING

Retrieving tuples in the order they appear in a non-clustered index is inefficient due to redundant reads.

A better approach is to find all the tuples that the query needs and then sort them based on their page ID.

The DBMS retrieves each page once.

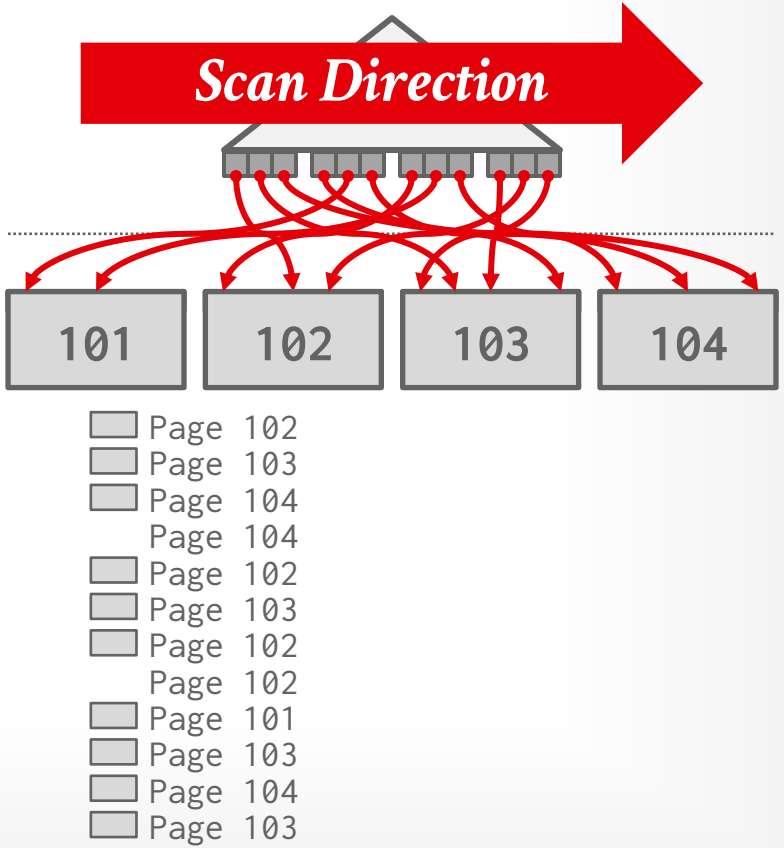


INDEX SCAN PAGE SORTING

Retrieving tuples in the order they appear in a non-clustered index is inefficient due to redundant reads.

A better approach is to find all the tuples that the query needs and then sort them based on their page ID.

The DBMS retrieves each page once.

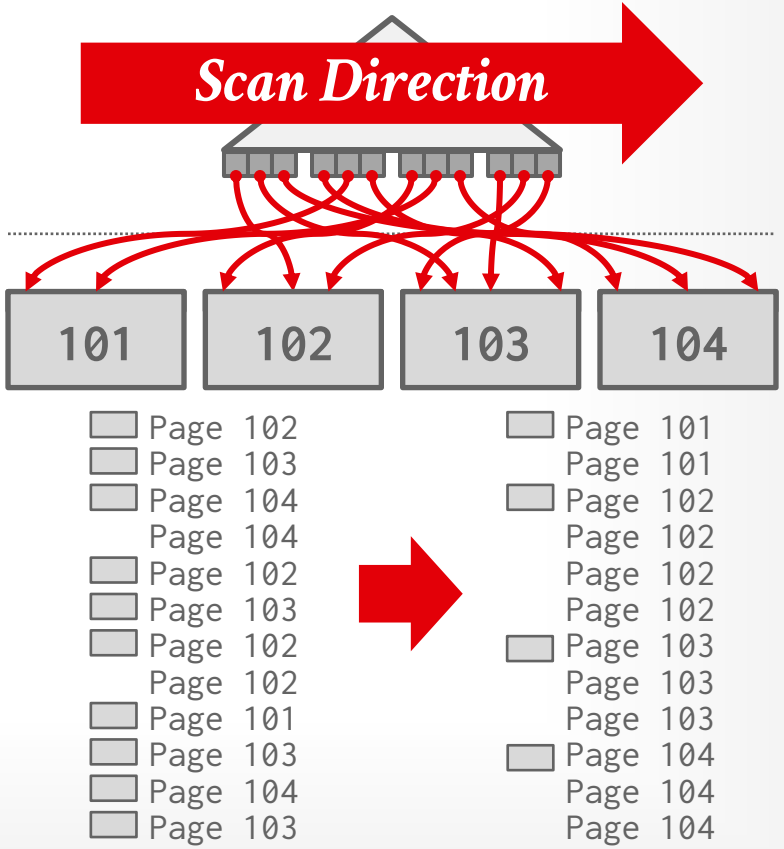


INDEX SCAN PAGE SORTING

Retrieving tuples in the order they appear in a non-clustered index is inefficient due to redundant reads.

A better approach is to find all the tuples that the query needs and then sort them based on their page ID.

The DBMS retrieves each page once.



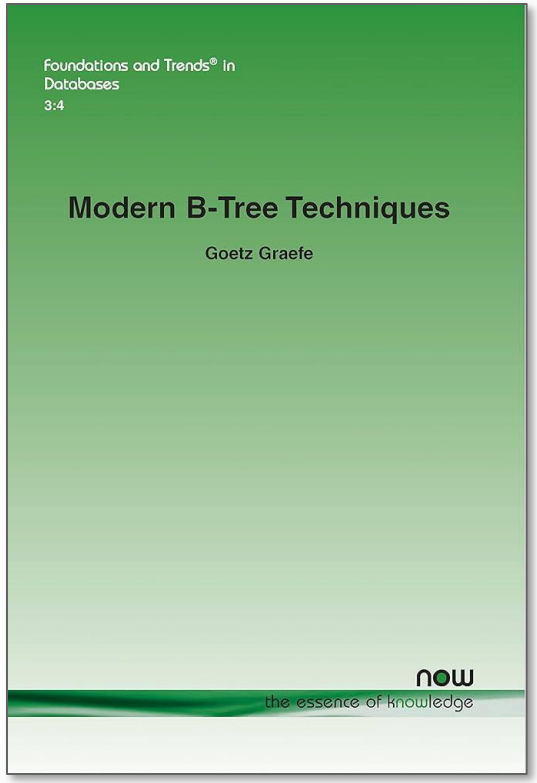
B+TREE DESIGN CHOICES

Node Size

Merge Threshold

Variable-Length Keys

Intra-Node Search



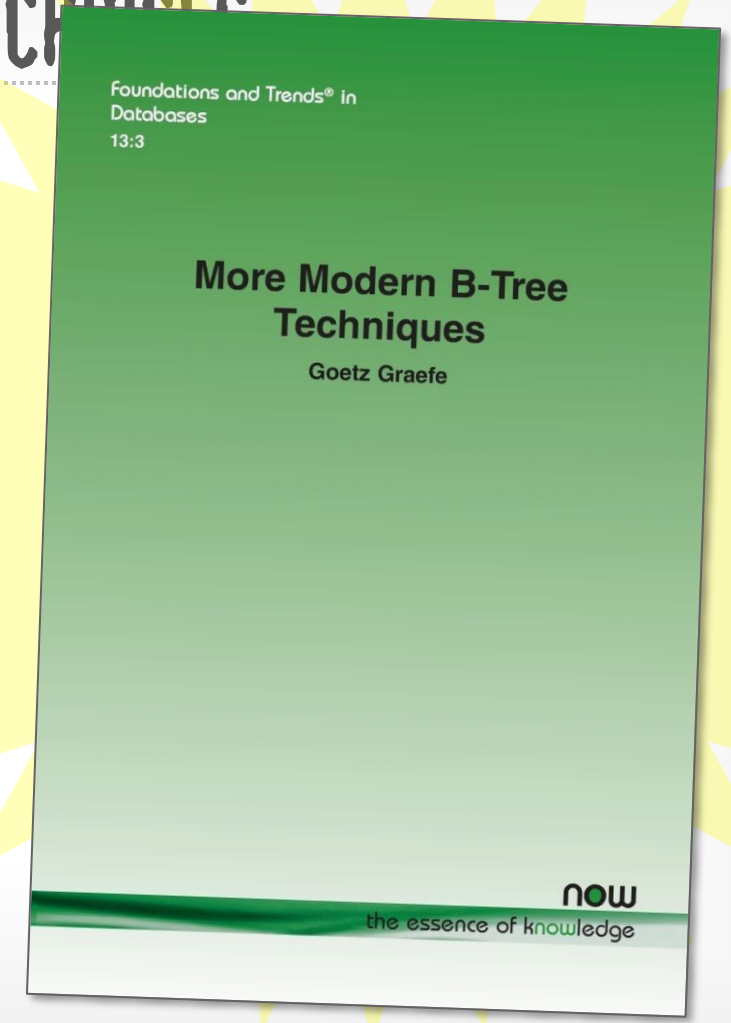
B+TREE DESIGN CHOICES

Node Size

Merge Threshold

Variable-Length Keys

Intra-Node Search



NODE SIZE

The slower the storage device, the larger the optimal node size for a B+Tree.

→ HDD: ~1MB
→ SSD: ~10KB
→ In-Memory: ~512B

Optimal sizes can vary depending on the workload

→ Leaf Node Scans vs. Root-to-Leaf Traversals

MERGE THRESHOLD

Some DBMSs do not always merge nodes when they are half full.

→ Average occupancy rate for B+Tree nodes is 69%.

Delaying a merge operation may reduce the amount of reorganization.

It may also be better to let underfilled nodes exist and then periodically rebuild entire tree.

This is why PostgreSQL calls their B+Tree a "non-balanced" B+Tree (nbtree).

VARIABLE-LENGTH KEYS

Approach #1: Pointers

→ Store the keys as pointers to the tuple's attribute.
→ Also called T-Trees (in-memory DBMSs)

Approach #2: Variable-Length Nodes

→ The size of each node in the index can vary.
→ Requires careful memory management.

Approach #3: Padding

→ Always pad the key to be max length of the key type.

Approach #4: Key Map / Indirection

→ Embed an array of pointers that map to the key + value list within the node.

INTRA-NODE SEARCH

Approach #1: Linear

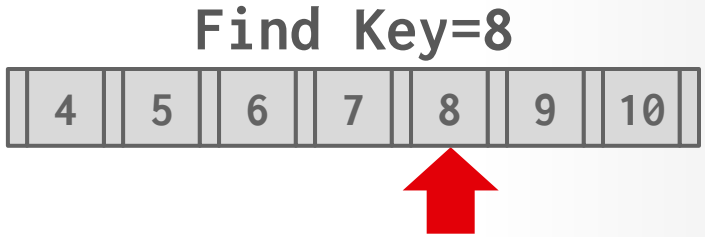
→ Scan node keys from beginning to end.
→ Use SIMD to vectorize comparisons.

INTRA-NODE SEARCH

Approach #1: Linear

→ Scan node keys from beginning to end.

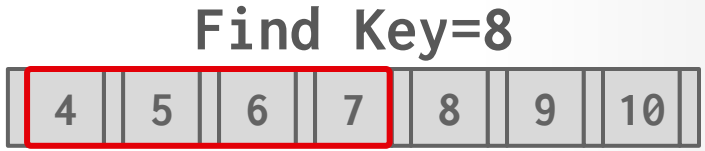
→ Use SIMD to vectorize comparisons.



INTRA-NODE SEARCH

Approach #1: Linear

→ Scan node keys from beginning to end.
→ Use SIMD to vectorize comparisons.

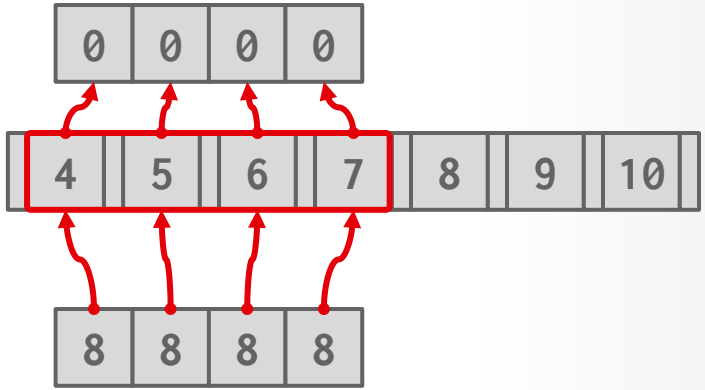



```
_mm_cmpeq_epi32_mask(a, b)
```

INTRA-NODE SEARCH

Approach #1: Linear

→ Scan node keys from beginning to end.
→ Use SIMD to vectorize comparisons.

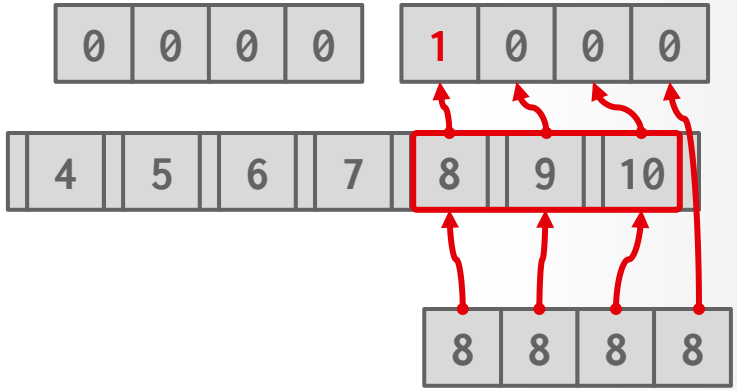



```
_mm_cmpeq_epi32_mask(a, b)
```

INTRA-NODE SEARCH

Approach #1: Linear

→ Scan node keys from beginning to end.
→ Use SIMD to vectorize comparisons.




```
_mm_cmpeq_epi32_mask(a, b)
```

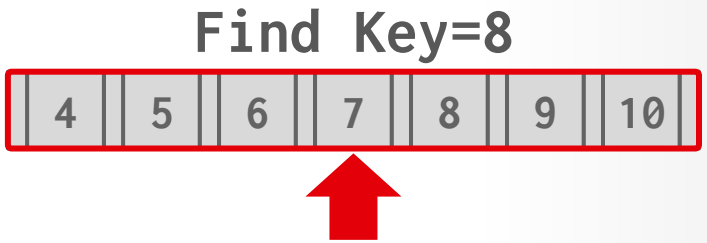
INTRA-NODE SEARCH

Approach #1: Linear

→ Scan node keys from beginning to end.
→ Use SIMD to vectorize comparisons.

Approach #2: Binary

→ Jump to middle key, pivot left/right depending on comparison.



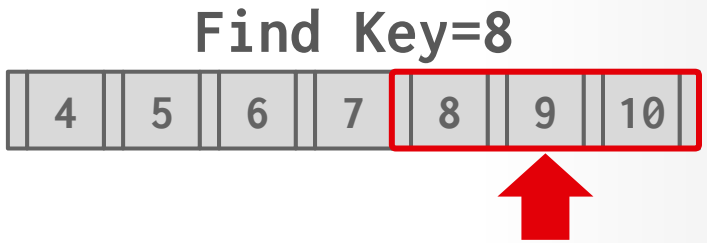
INTRA-NODE SEARCH

Approach #1: Linear

→ Scan node keys from beginning to end.
→ Use SIMD to vectorize comparisons.

Approach #2: Binary

→ Jump to middle key, pivot left/right depending on comparison.



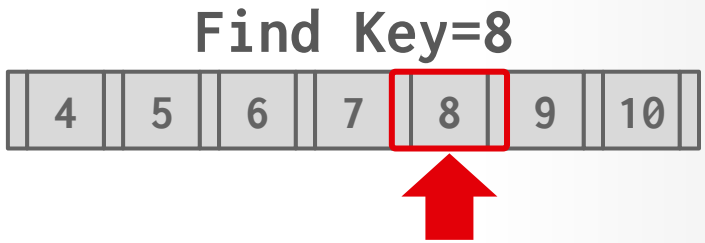
INTRA-NODE SEARCH

Approach #1: Linear

→ Scan node keys from beginning to end.
→ Use SIMD to vectorize comparisons.

Approach #2: Binary

→ Jump to middle key, pivot left/right depending on comparison.



INTRA-NODE SEARCH

Approach #1: Linear

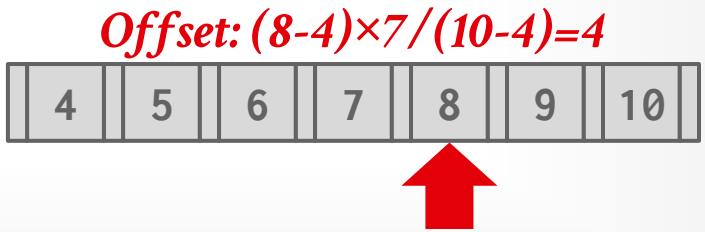
→ Scan node keys from beginning to end.
→ Use SIMD to vectorize comparisons.

Approach #2: Binary

→ Jump to middle key, pivot left/right depending on comparison.

A

→ Approximate location of desired key based on known distribution of keys.



OPTIMIZATIONS

Pointer Swizzling

Buffered Updates (*Bε-trees*)

Partial Indexes

Include Columns

Prefix Compression

Deduplication

Suffix Truncation

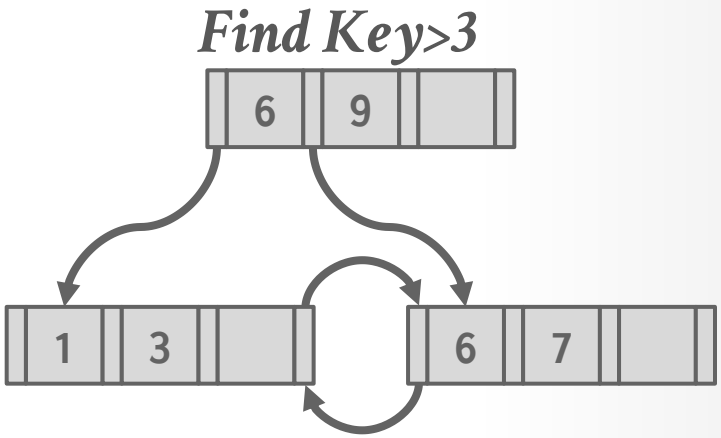
Bulk Insert

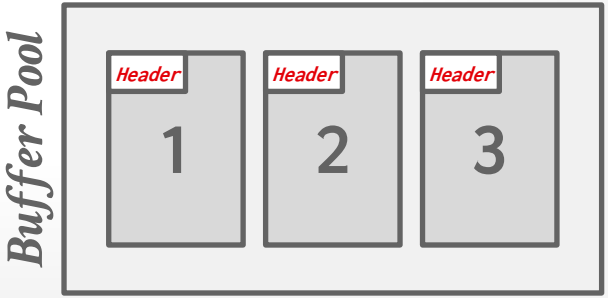
Many more…

POINTER SWIZZLING

Nodes use page ids to reference other nodes in the index. The DBMS must get the memory location from the page table during traversal.

If a page is pinned in the buffer pool, then we can store raw pointers instead of page ids. This avoids address lookups from the page table.

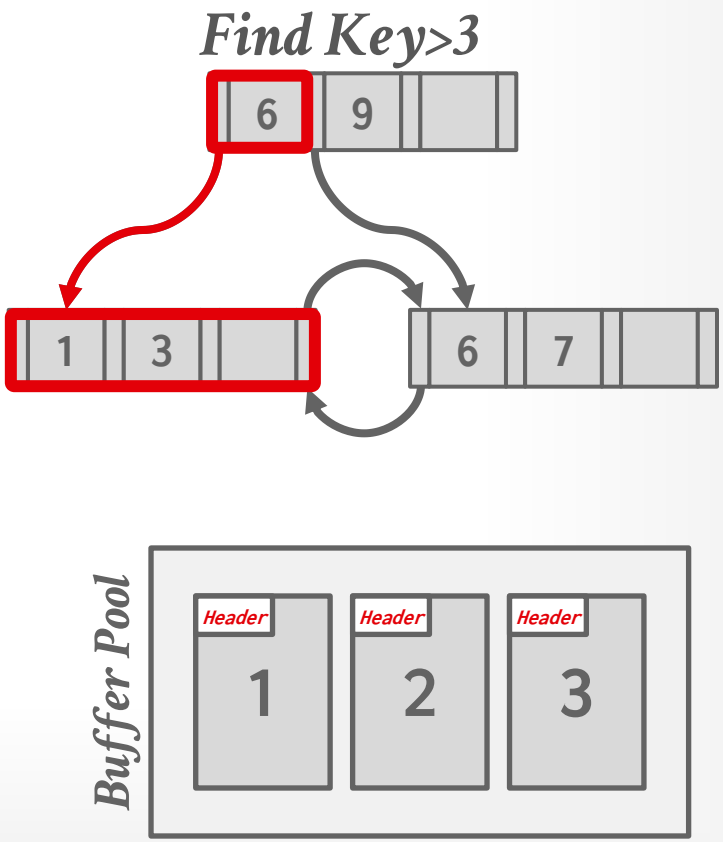




POINTER SWIZZLING

Nodes use page ids to reference other nodes in the index. The DBMS must get the memory location from the page table during traversal.

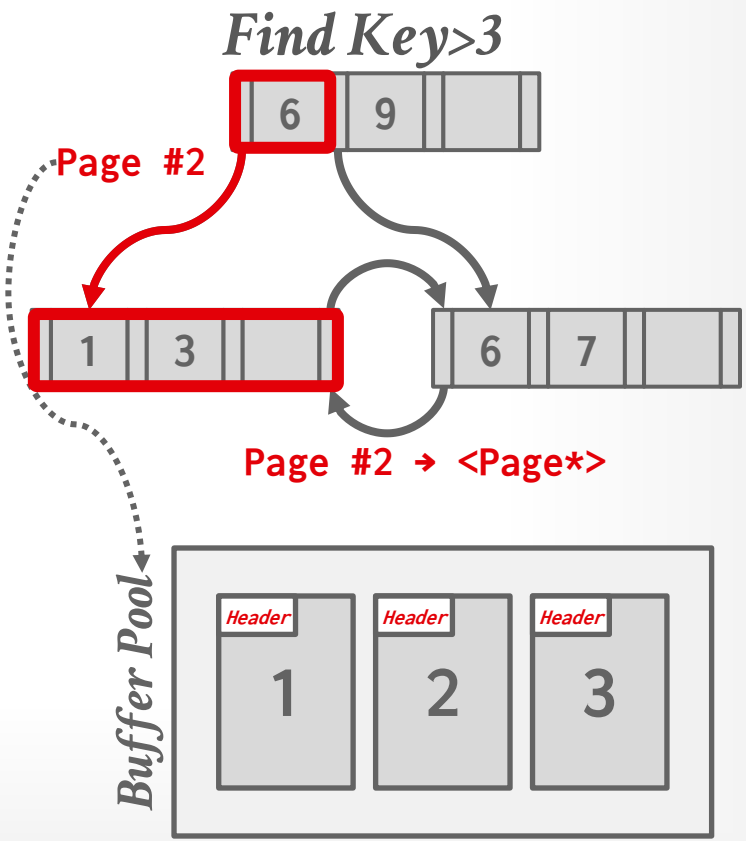
If a page is pinned in the buffer pool, then we can store raw pointers instead of page ids. This avoids address lookups from the page table.



POINTER SWIZZLING

Nodes use page ids to reference other nodes in the index. The DBMS must get the memory location from the page table during traversal.

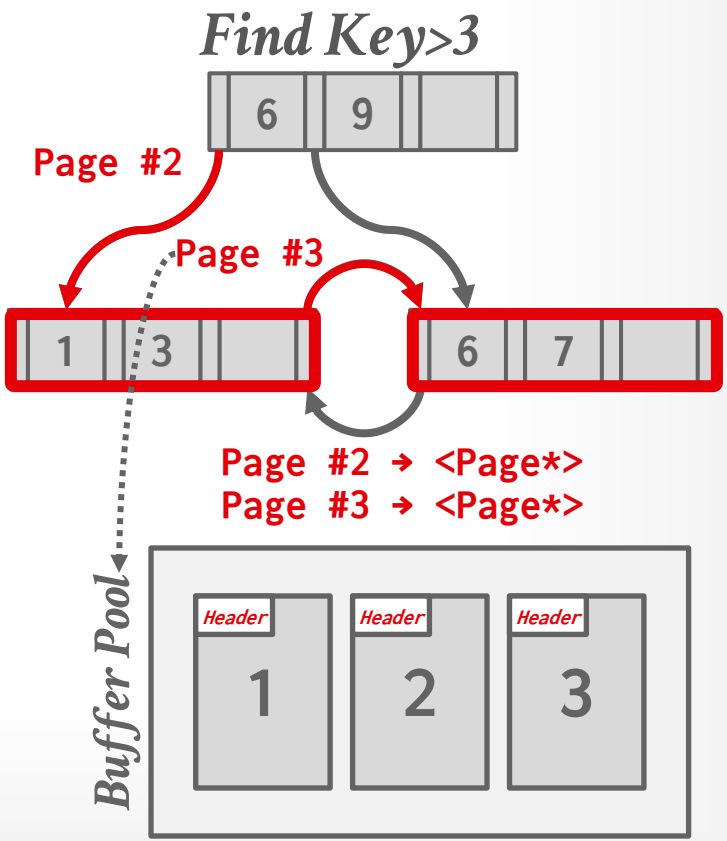
If a page is pinned in the buffer pool, then we can store raw pointers instead of page ids. This avoids address lookups from the page table.



POINTER SWIZZLING

Nodes use page ids to reference other nodes in the index. The DBMS must get the memory location from the page table during traversal.

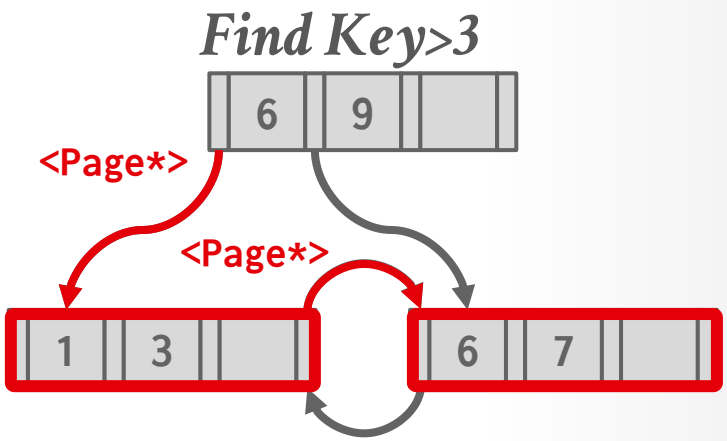
If a page is pinned in the buffer pool, then we can store raw pointers instead of page ids. This avoids address lookups from the page table.

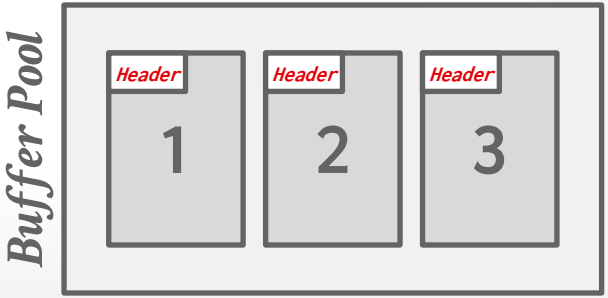


POINTER SWIZZLING

Nodes use page ids to reference other nodes in the index. The DBMS must get the memory location from the page table during traversal.

If a page is pinned in the buffer pool, then we can store raw pointers instead of page ids. This avoids address lookups from the page table.





OBSERVATION

Modifying a B+tree is expensive when the DBMS has to split/merge nodes.

→ Worst case is when DBMS reorganizes the entire tree.

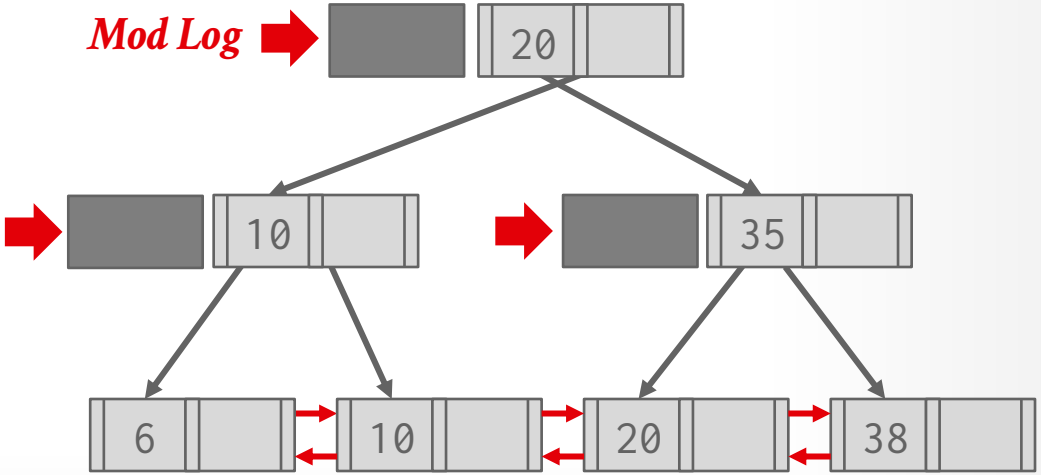
→ The worker that causes a split/merge is responsible for doing the work.

What if there was a way to delay updates and then apply multiple changes together in a batch?

WRITE-OPTIMIZED B+TREE

Instead of immediately applying updates, store changes to key/value entries in log buffers at inner nodes.
→ aka **Fractal Trees** / **Bε-trees**.

Updates cascade down to lower nodes incrementally when buffers get full.



Tokutek SPLINTERDB RelationalAI ChromoDB STSDB

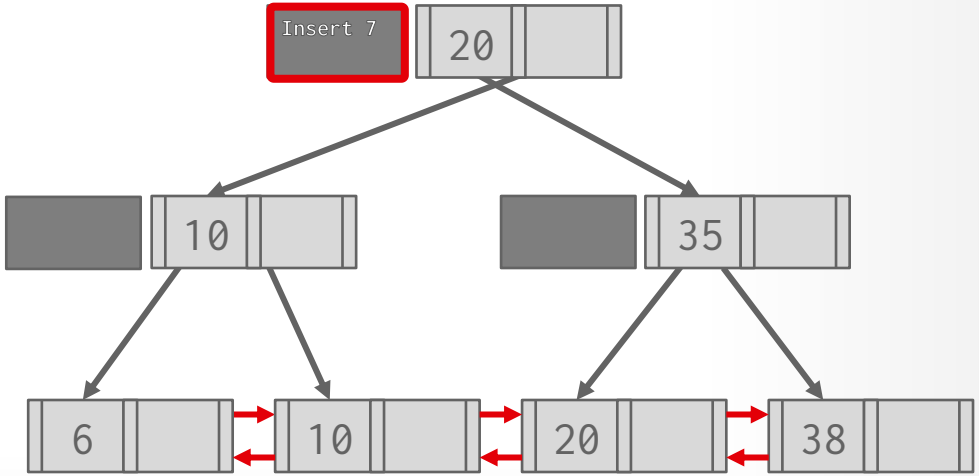
WRITE-OPTIMIZED B+TREE

Instead of immediately applying updates, store changes to key/value entries in log buffers at inner nodes.
→ aka **Fractal Trees** / **Bε-trees**.

Updates cascade down to lower nodes incrementally when buffers get full.

Tokutek SPLINTERDB
RelationalAI ChromoDB
STSDB

Insert 7

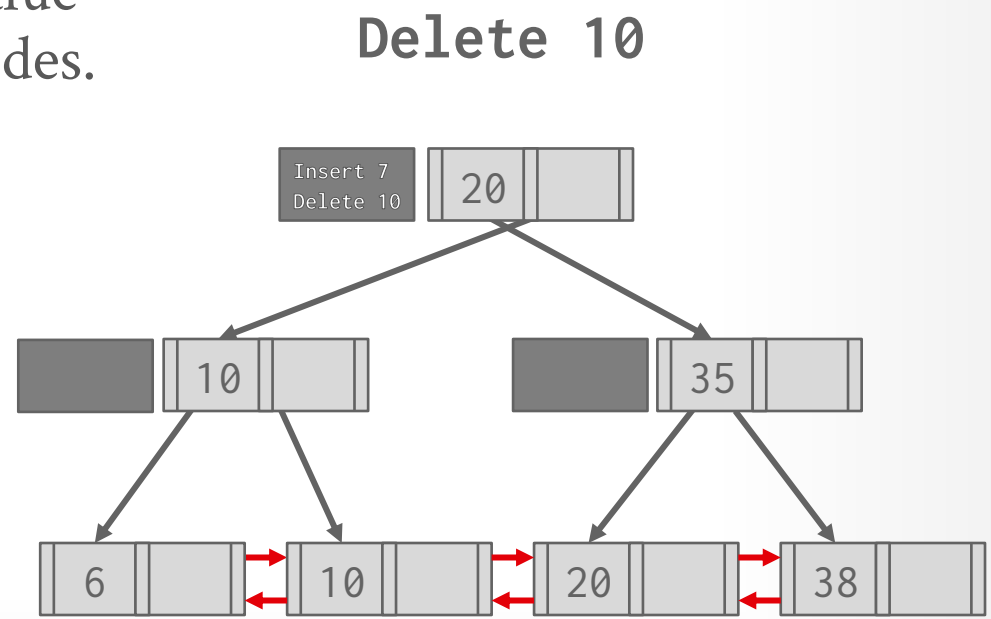


WRITE-OPTIMIZED B+TREE

Instead of immediately applying updates, store changes to key/value entries in log buffers at inner nodes.
→ aka **Fractal Trees** / **Bε-trees**.

Updates cascade down to lower nodes incrementally when buffers get full.

Tokutek SPLINTERDB
RelationalAI ChromoDB
STSDB



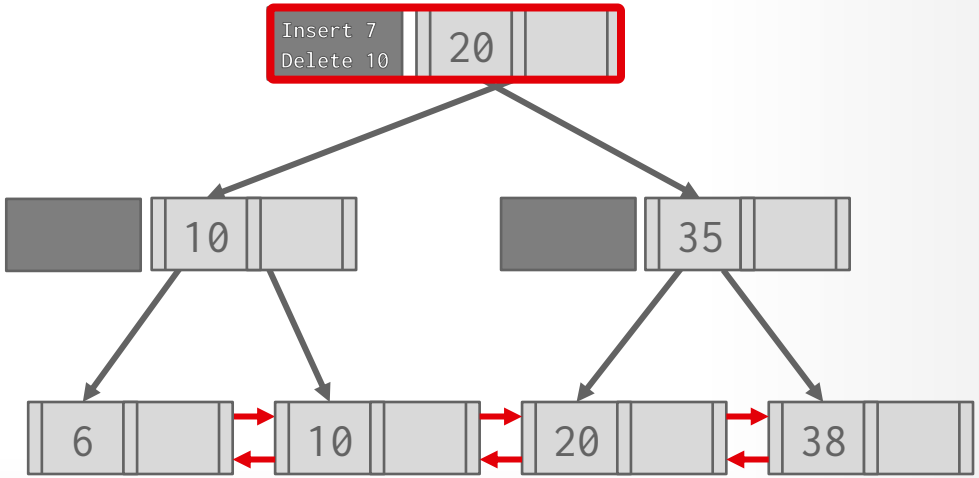
WRITE-OPTIMIZED B+TREE

Instead of immediately applying updates, store changes to key/value entries in log buffers at inner nodes.
→ aka **Fractal Trees** / **Bε-trees**.

Updates cascade down to lower nodes incrementally when buffers get full.

Tokutek SPLINTERDB
RelationalAI ChromoDB
STSDB

Find 10



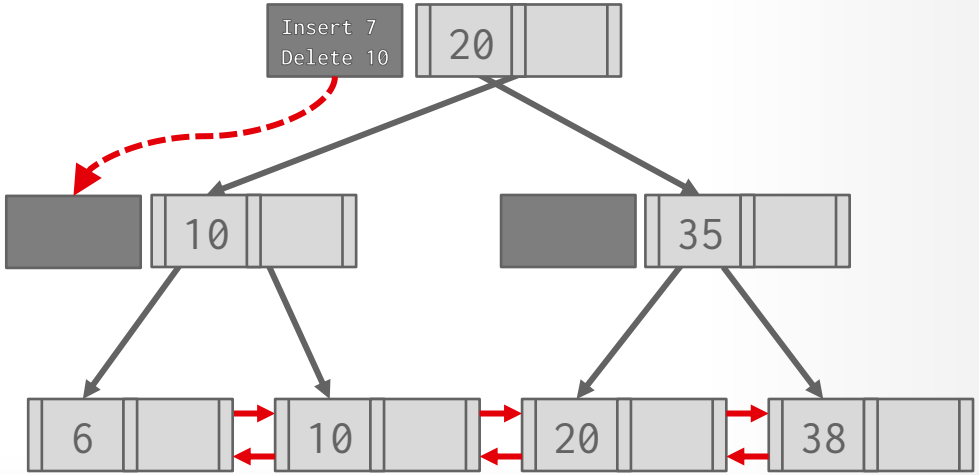
WRITE-OPTIMIZED B+TREE

Instead of immediately applying updates, store changes to key/value entries in log buffers at inner nodes.
→ aka **Fractal Trees** / **Bε-trees**.

Updates cascade down to lower nodes incrementally when buffers get full.

Tokutek SPLINTERDB
RelationalAI ChromoDB
STSDB

Insert 40



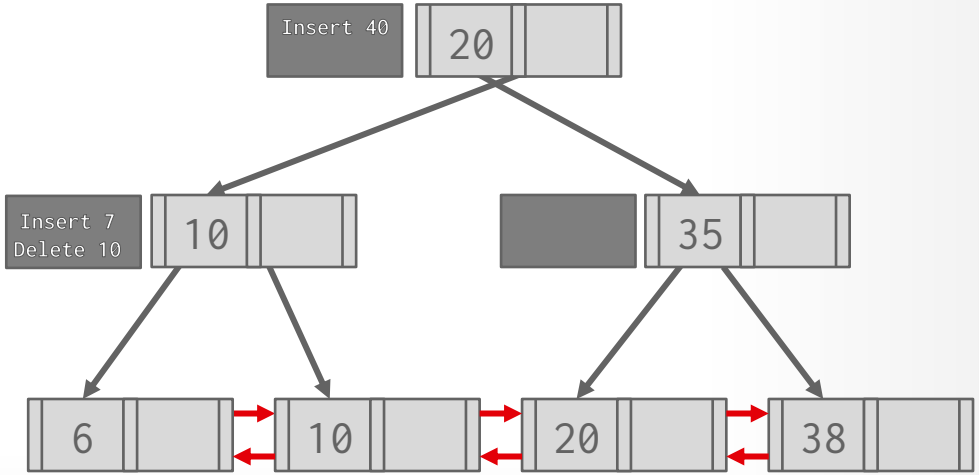
WRITE-OPTIMIZED B+TREE

Instead of immediately applying updates, store changes to key/value entries in log buffers at inner nodes.
→ aka **Fractal Trees** / **Bε-trees**.

Updates cascade down to lower nodes incrementally when buffers get full.

Tokutek SPLINTERDB
RelationalAI ChromoDB
STSDB

Insert 40



PARTIAL INDEXES

Create an index on a subset of the entire table. This potentially reduces its size and the amount of overhead to maintain it.

```
CREATE INDEX idx_foo
          ON foo (a, b)
       WHERE c = 'WuTang';
```

One common use case is to partition indexes by date ranges.
→ Create a separate index per month, year.

PARTIAL INDEXES

Create an index on a subset of the entire table. This potentially reduces its size and the amount of overhead to maintain it.

```
CREATE INDEX idx_foo
          ON foo (a, b)
       WHERE c = 'WuTang';
```

One common use case is to partition indexes by date ranges.

→ Create a separate index per month, year.

PARTIAL INDEXES

Create an index on a subset of the entire table. This potentially reduces its size and the amount of overhead to maintain it.

```
CREATE INDEX idx_foo
          ON foo (a, b)
       WHERE c = 'WuTang';
```

One common use case is to partition indexes by date ranges.
→ Create a separate index per month, year.

```
SELECT b FROM foo
 WHERE a = 123
   AND c = 'WuTang';
```

PARTIAL INDEXES

Create an index on a subset of the entire table. This potentially reduces its size and the amount of overhead to maintain it.

One common use case is to partition indexes by date ranges.
→ Create a separate index per month, year.

```
CREATE INDEX idx_foo
          ON foo (a, b)
       WHERE c = 'WuTang';
```

```
SELECT b FROM foo
 WHERE a = 123
   AND c = 'WuTang';
```

PARTIAL INDEXES

Create an index on a subset of the entire table. This potentially reduces its size and the amount of overhead to maintain it.

One common use case is to partition indexes by date ranges.
→ Create a separate index per month, year.

```
CREATE INDEX idx_foo
          ON foo (a, b)
       WHERE c = 'WuTang';
```

```
SELECT b FROM foo
 WHERE a = 123
```

INDEX INCLUDE COLUMNS

Embed additional columns in indexes to support index-only queries.

These extra columns are only stored in the leaf nodes and are not part of the search key.

```
CREATE INDEX idx_foo
          ON foo (a, b)
     INCLUDE (c);
```

INDEX INCLUDE COLUMNS

Embed additional columns in indexes to support index-only queries.

These extra columns are only stored in the leaf nodes and are not part of the search key.

```
CREATE INDEX idx_foo
          ON foo (a, b)
     INCLUDE (c);
```

```
SELECT b FROM foo
 WHERE a = 123
   AND c = 'WuTang';
```

INDEX INCLUDE COLUMNS

Embed additional columns in indexes to support index-only queries.

These extra columns are only stored in the leaf nodes and are not part of the search key.

```
CREATE INDEX idx_foo
        ON foo (a, b)
     INCLUDE (c);
```

```
SELECT b FROM foo
 WHERE a = 123
   AND c = 'WuTang';
```

INDEX INCLUDE COLUMNS

Embed additional columns in indexes to support index-only queries.

These extra columns are only stored in the leaf nodes and are not part of the search key.

```
CREATE INDEX idx_foo
        ON foo (a, b)
   INCLUDE (c);

SELECT b FROM foo
 WHERE a = 123
   AND c = 'WuTang';
```

INDEX INCLUDE COLUMNS

Embed additional columns in indexes to support index-only queries.

These extra columns are only stored in the leaf nodes and are not part of the search key.

If all attributes a query needs are available in an index, then the DBMS does not need to retrieve the tuple.
→ AKA covering index or index-only scans.

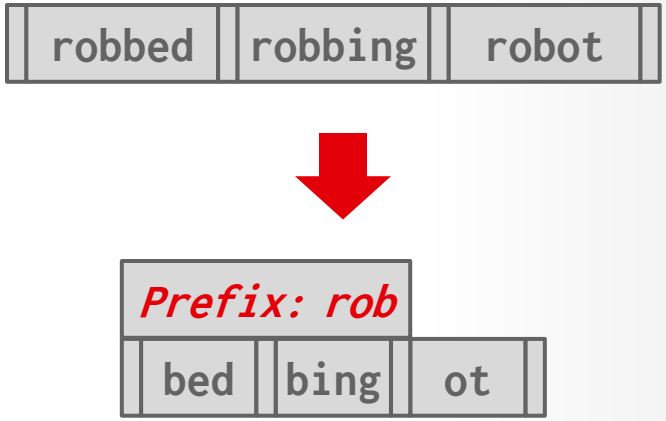
```
CREATE INDEX idx_foo
        ON foo (a, b)
   INCLUDE (c);

SELECT b FROM foo
 WHERE a = 123
   AND c = 'WuTang';
```

PREFIX COMPRESSION

Sorted keys in the same leaf node are likely to have the same prefix.

Instead of storing the entire key each time, extract common prefix and store only unique suffix for each key.
→ Many variations.



DEDUPLICATION

Non-unique indexes can end up storing multiple copies of the same key in leaf nodes.

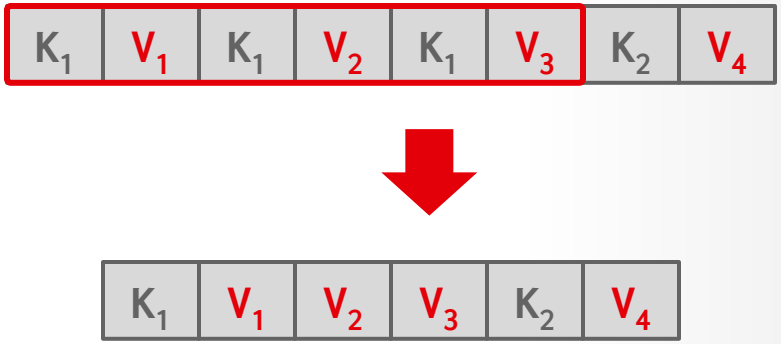
The leaf node can store the key once and then maintain a "posting list" of tuples with that key (similar to what we discussed for hash tables).

K_1	V_1	K_1	V_2	K_1	V_3	K_2	V_4

DEDUPLICATION

Non-unique indexes can end up storing multiple copies of the same key in leaf nodes.

The leaf node can store the key once and then maintain a "posting list" of tuples with that key (similar to what we discussed for hash tables).

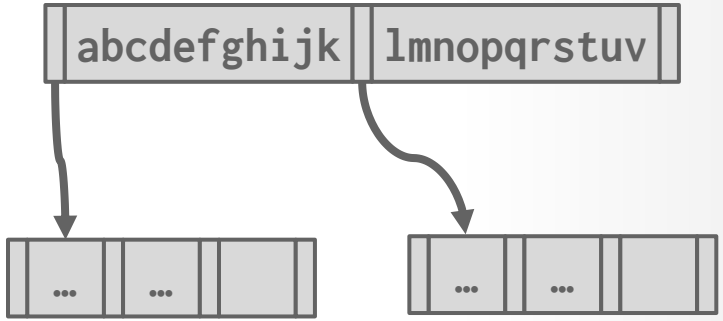


SUFFIX TRUNCATION

The keys in the inner nodes are only used to "direct traffic".

→ We don't need the entire key.

Store a minimum prefix that is needed to correctly route probes into the index.

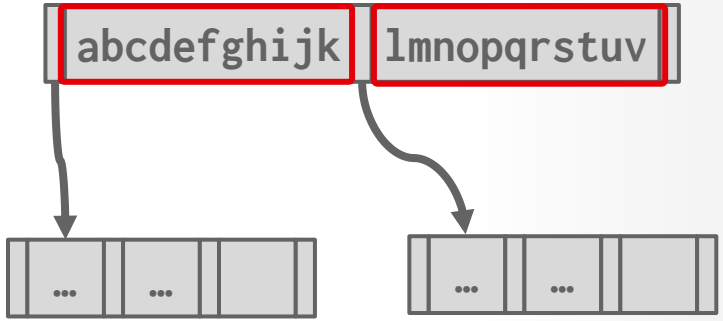


SUFFIX TRUNCATION

The keys in the inner nodes are only used to "direct traffic".

→ We don't need the entire key.

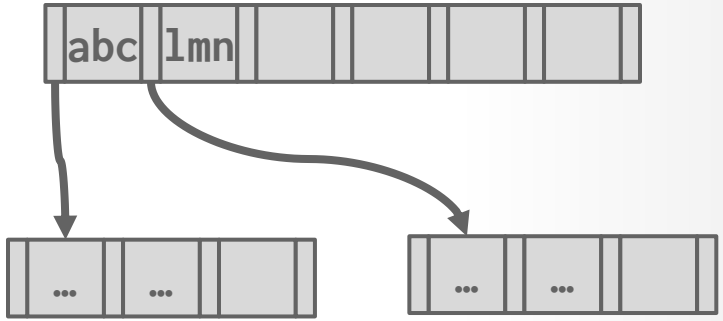
Store a minimum prefix that is needed to correctly route probes into the index.



SUFFIX TRUNCATION

The keys in the inner nodes are only used to "direct traffic".
→ We don't need the entire key.

Store a minimum prefix that is needed to correctly route probes into the index.

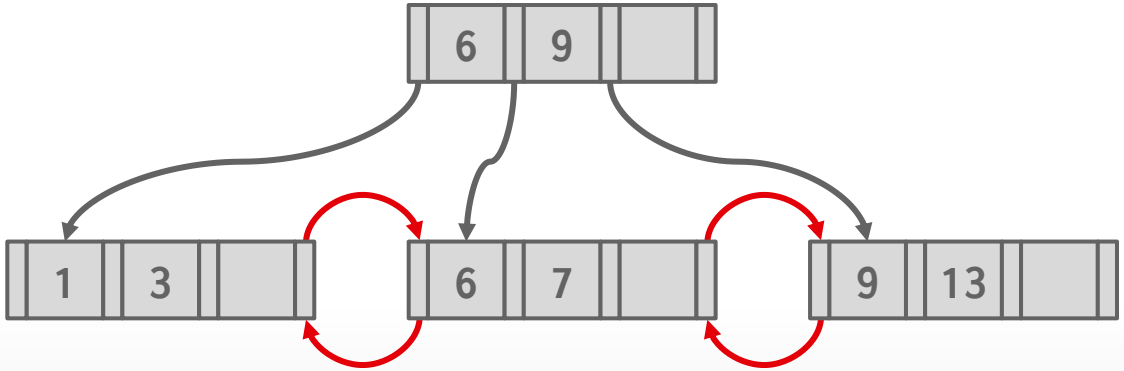


BULK INSERT

The fastest way to build a new B+Tree for an existing table is to first sort the keys and then build the index from the bottom up.

Keys: 3, 7, 9, 13, 6, 1

Sorted Keys: 1, 3, 6, 7, 9, 13



CONCLUSION

The venerable B+Tree is (almost) always a good choice for your DBMS.

NEXT CLASS

Bloom Filters

Tries / Radix Trees / Patricia Trees

Skip Lists

Inverted Indexes

Vector Indexes